



UMCS

UNIwersytet Marii Curie-Skłodowskiej
w Lublinie

Wydział Matematyki, Fizyki i Informatyki

Kierunek: **Geoinformatyka**

Specjalność:

Krystian Banach

Nr albumu: 307619

Projekt i implementacja oprogramowania do analizy i przetwarzania tras GPX na potrzeby zawodów sportowych

Design and implementation of software for analyzing and
processing GPX routes for sports competitions

Praca magisterska
napisana w Katedrze Oprogramowania Systemów Informatycznych
Instytutu Informatyki
pod kierunkiem dr Michała Klisowskiego

Lublin 2026

Spis treści

Wstęp	5
1 Teoretyczne podstawy analizy tras GPX	7
1.1 Pamięć i wydajność obliczeń	7
1.1.1 Pamięć podręczna i hierarchia pamięci	8
1.1.2 Równoległe wykonywanie operacji	9
1.1.3 Wektoryzacja	9
1.2 Dane GPS i format GPX	10
1.2.1 Charakterystyka danych GPS	10
1.2.2 Struktura pliku GPX	11
1.2.3 Układy współrzędnych w analizie tras	12
1.3 Metody analizy i porównywania tras	13
1.3.1 Obliczanie dystansu i przewyższeń	14
1.3.2 Porównywanie trasy z trasą referencyjną	16
1.3.3 Dyskretna odległość Frécheta	17
2 Reprezentacja danych i narzędzia obliczeniowe w Pythonie	19
2.1 Python i reprezentacja danych liczbowych	19
2.1.1 Listy Pythona jako wariant bazowy	20
2.1.2 NumPy i tablice numeryczne	21
2.1.3 Numba i kompilacja JIT	23
3 Projekt i implementacja oprogramowania do analizy tras GPX	25
3.1 Struktura projektu	25
3.2 Implementacja parsera plików GPX	27
3.3 Implementacja operacji analizy tras	30
3.3.1 Obliczanie dystansu i przewyższeń	30
3.3.2 Porównywanie trasy zawodnika z trasą referencyjną	35
3.3.3 Obliczanie dyskretnej odległości Frécheta	43
3.4 Implementacja benchmarków	48

4	Metodyka eksperymentów i analiza wyników	53
4.1	Środowisko testowe	53
4.2	Metodyka pomiarów	54
4.3	Wyniki benchmarków	55
4.3.1	Obliczanie dystansu	55
4.3.2	Obliczanie przewyższeń	57
4.3.3	Porównywanie trasy zawodnika z trasą referencyjną	59
4.3.4	Dyskretna odległość Frécheta	61
4.4	Wnioski	63
4.5	Podsumowanie	66
	Bibliografia	67
	Spis listingów	71
	Spis rysunków	73
	Spis tabel	75

Wstęp

Rejestrowanie aktywności sportowych za pomocą zegarków, liczników rowerowych i innych odbiorników GPS stało się powszechną praktyką zarówno w treningu amatorskim, jak i w organizacji zawodów sportowych. Efektem takiego pomiaru jest cyfrowy ślad trasy, czyli uporządkowana sekwencja punktów opisujących przebieg ruchu użytkownika. Dane te obejmują przede wszystkim kolejne położenia geograficzne, a często również wysokość, czas pomiaru oraz dodatkowe informacje związane z aktywnością. Jednym z powszechnie stosowanych formatów wymiany takich danych jest GPX, czyli tekstowy format oparty na XML, przeznaczony do zapisu punktów, tras i śladów geograficznych.

Analiza tras GPX może obejmować zarówno podstawowe operacje wykonywane na pojedynczym śladzie, jak i bardziej złożone metody porównywania dwóch przebiegów. Do pierwszej grupy należą między innymi obliczanie długości trasy oraz sumy podejść i zejść. Do drugiej grupy można zaliczyć porównywanie śladu zawodnika z trasą referencyjną, na przykład w celu określenia, czy zawodnik poruszał się w pobliżu wyznaczonego przebiegu trasy. Tego typu operacje mogą mieć znaczenie w analizie aktywności sportowych, planowaniu tras oraz weryfikacji zgodności rzeczywistego śladu z trasą wzorcową.

Dane GPX mają charakter sekwencyjny i liczbowy, ponieważ ślad trasy składa się z kolejnych punktów opisanych współrzędnymi geograficznymi oraz, w zależności od pliku, dodatkowymi wartościami takimi jak wysokość lub czas pomiaru. Przetwarzanie takich danych wymaga wielokrotnego wykonywania podobnych operacji na kolejnych elementach sekwencji. Wraz ze wzrostem liczby punktów znaczenia nabiera nie tylko dobór samej metody obliczeniowej, lecz także sposób reprezentacji danych oraz sposób wykonania pętli w programie.

Celem pracy jest zaprojektowanie i zaimplementowanie programu do analizy oraz przetwarzania tras zapisanych w formacie GPX, a następnie porównanie wybranych wariantów obliczeniowych pod względem czasu wykonania. W pracy uwzględniono wariant bazowy oparty na listach Pythona, wariant wykorzystujący tablice NumPy oraz wariant wykorzystujący bibliotekę Numba i kompilację JIT. Porównanie tych podejść pozwala sprawdzić, w jakich przypadkach zapis tablicowy lub kompilacja pętli numerycznych może skrócić czas wykonania, a w jakich przypadkach struktura algorytmu ogranicza możliwość uzyskania przyspieszenia.

Zakres pracy obejmuje przygotowanie parsera plików GPX, implementację funkcji obliczających dystans i przewyższenia, metodę porównywania punktów śladu zawodnika z segmentami trasy referencyjnej oraz funkcję wyznaczającą dyskretną odległość Fréchet. Uwzględniono również transformację współrzędnych geograficznych do układu metrycznego w metodach wymagających obliczeń geometrycznych w płaszczyźnie.

Praca ma charakter implementacyjno-eksperymentalny. Oprócz przygotowania programu obejmuje przeprowadzenie benchmarków porównujących wariant listowy, wariant NumPy oraz wariant Numba dla operacji o różnej strukturze obliczeniowej. Dane testowe zostały wygenerowane syntetycznie, aby możliwe było kontrolowanie liczby punktów i powtarzalne badanie czasu wykonania funkcji dla rosnących rozmiarów danych wejściowych.

Praca została podzielona na cztery główne rozdziały. W pierwszym rozdziale przedstawiono podstawy teoretyczne analizy tras GPX, w tym charakter danych wejściowych, strukturę formatu GPX, znaczenie układów współrzędnych oraz wybrane metody analizy i porównywania tras. Drugi rozdział dotyczy reprezentacji danych i narzędzi obliczeniowych w języku Python, ze szczególnym uwzględnieniem list Pythona, tablic NumPy oraz biblioteki Numba.

W trzecim rozdziale opisano projekt i implementację programu, obejmującą parser GPX, funkcje analityczne oraz skrypty benchmarkowe. Czwarty rozdział zawiera metodykę pomiarów, wyniki benchmarków oraz omówienie ograniczeń przeprowadzonych analiz.

Rozdział 1

Teoretyczne podstawy analizy tras GPX

Rozdział przedstawia podstawy teoretyczne potrzebne do omówienia projektu oraz implementacji programu analizującego trasy zapisane w formacie GPX. Punktem wyjścia jest charakterystyka danych wejściowych, czyli śladów GPS złożonych z kolejnych punktów opisujących położenie użytkownika. Następnie omówiono strukturę formatu GPX, znaczenie układów współrzędnych oraz metody wykorzystywane do obliczania i porównywania tras.

W rozdziale uwzględniono również wybrane zagadnienia związane z wydajnością obliczeń. Przetwarzanie tras GPX wymaga wielokrotnego wykonywania operacji na sekwencjach wartości liczbowych, dlatego znaczenie mają nie tylko same metody analityczne, lecz także sposób reprezentacji danych i możliwość zapisu części obliczeń jako operacji tablicowych.

1.1 Pamięć i wydajność obliczeń

Wydajność programów przetwarzających duże zbiory danych zależy nie tylko od liczby wykonywanych operacji arytmetycznych, lecz także od sposobu organizacji dostępu do pamięci. W wielu zastosowaniach numerycznych czas wykonania może być ograniczany nie przez samą liczbę operacji procesora, ale przez koszt dostarczenia danych z pamięci operacyjnej. Z tego powodu współczesne architektury komputerowe wykorzystują hierarchię pamięci obejmującą między innymi rejestry, pamięć podręczną oraz pamięć główną [4, 5]. Efektywność wykorzystania tej hierarchii zależy między innymi od lokalności czasowej i przestrzennej odwołań do danych.[5]

Trasy zapisane w formacie GPX mogą być przetwarzane jako uporządkowane sekwencje punktów śladu `trkpt`. [3] Operacje takie jak obliczanie dystansu, sumowanie przewyższeń lub porównywanie śladu z trasą referencyjną wymagają wielokrotnego przechodzenia

po kolejnych wartościach liczbowych. Z tego powodu sposób reprezentacji danych może wpływać zarówno na koszt dostępu do pamięci, jak i na możliwość zapisu obliczeń w postaci operacji tablicowych.

1.1.1 Pamięć podręczna i hierarchia pamięci

Pamięć podręczna procesora pełni funkcję szybkiej warstwy pośredniczącej pomiędzy jednostką centralną a pamięcią operacyjną. Jej zastosowanie wynika z różnicy prędkości pomiędzy procesorem a pamięcią główną: dane używane niedawno lub potencjalnie potrzebne w najbliższym czasie mogą zostać umieszczone bliżej procesora.[4] Dzięki temu część odwołań do danych może zostać obsłużona bez każdorazowego pobierania ich z pamięci głównej.

Działanie pamięci podręcznej jest związane z pojęciami trafienia oraz chybienia. Trafienie cache występuje wtedy, gdy poszukiwane dane znajdują się już w pamięci podręcznej i mogą zostać z niej odczytane. Chybiecie oznacza natomiast konieczność pobrania danych z niższego poziomu hierarchii pamięci i umieszczenia ich w cache.[5]

Podstawą efektywnego wykorzystania pamięci podręcznej jest lokalność odwołań. Lokalność czasowa oznacza możliwość ponownego użycia danych wykorzystywanych niedawno, natomiast lokalność przestrzenna dotyczy korzystania z danych położonych blisko siebie w przestrzeni adresowej.[5] W praktyce czas wykonania programu może być ograniczany zarówno przez koszt operacji arytmetycznych, jak i przez koszt dostępu do danych. W niniejszej pracy zagadnienie to stanowi jedynie kontekst dla porównania list Pythona z jednorodnymi tablicami numerycznymi.

Istotną cechą pamięci podręcznej jest blokowy sposób przenoszenia danych między pamięcią główną a cache. Pamięć główna jest traktowana jako zbiór bloków adresowalnych komórek, natomiast pamięć podręczna składa się z wierszy odpowiadających takim blokom.[5] Odczyt jednej wartości może więc spowodować przeniesienie do pamięci podręcznej również wartości sąsiednich, co sprzyja programom przetwarzającym dane ułożone kolejno w pamięci.

Z tej perspektywy wybór struktury danych ma znaczenie nie tylko organizacyjne, lecz także wydajnościowe. Dane przechowywane w sposób ciągły, na przykład w jednorodnych tablicach liczbowych, mogą sprzyjać sekwencyjnemu odczytowi i lepiej odpowiadać sposobowi działania pamięci podręcznej [5]. Nie oznacza to jednak, że każda zmiana reprezentacji danych automatycznie prowadzi do skrócenia czasu wykonania. Efekt zależy od konkretnego algorytmu, sposobu dostępu do danych oraz narzutu związanego z używanym mechanizmem wykonania. Mniej korzystny z punktu widzenia lokalności pamięci może być model, w którym dane są reprezentowane jako wiele odrębnych obiektów powiązanych odwołaniami, ponieważ kolejne logiczne elementy przetwarzanej sekwencji nie muszą znajdować się obok siebie w pamięci. W języku Python dane programu są

reprezentowane jako obiekty posiadające tożsamość, typ i wartość.[6] Taki model jest istotny przy porównaniu list Pythona z jednorodnymi tablicami numerycznymi.

1.1.2 Równoległe wykonywanie operacji

Wydajność obliczeń numerycznych może być zwiększana nie tylko przez zmianę algorytmu, lecz także przez wykorzystanie równoległości dostępnej na poziomie sprzętu i sposobu wykonania programu. Jednym z klasycznych sposobów opisu architektur obliczeniowych jest taksonomia Flynna, dzieląca systemy według liczby strumieni instrukcji i danych. W kontekście obliczeń numerycznych szczególne znaczenie ma model SIMD, czyli *Single Instruction, Multiple Data*, w którym jedna instrukcja jest wykonywana dla wielu elementów danych.[10]

Model SIMD jest istotny wtedy, gdy ta sama operacja może zostać zastosowana do wielu wartości liczbowych. W analizie tras GPX taki charakter mają między innymi operacje na współrzędnych, wysokościach oraz różnicach między sąsiednimi punktami. Jednorodna reprezentacja danych i uporządkowany układ wartości w pamięci mogą sprzyjać przetwarzaniu wielu elementów według tego samego schematu obliczeń.

W taksonomii Flynna wyróżnia się również model MIMD, czyli *Multiple Instruction, Multiple Data*, w którym równoległość dotyczy zarówno strumieni instrukcji, jak i strumieni danych.[10] Taki model można odnieść na przykład do niezależnego przetwarzania wielu tras lub wielu plików GPX. Taki model można odnieść na przykład do niezależnego przetwarzania wielu tras lub wielu plików GPX. W praktyce jednak nie każda część programu może być wykonywana równoległe. Możliwe przyspieszenie jest ograniczone przez fragmenty obliczeń, które muszą pozostać sekwencyjne, co opisuje prawo Amdahla [12].

1.1.3 Wektoryzacja

Wektoryzację można rozumieć jako sposób zapisu lub wykonania obliczeń, w którym ta sama operacja jest stosowana do wielu elementów danych zamiast do pojedynczych wartości przetwarzanych osobno. Pojęcie to może odnosić się zarówno do poziomu sprzętowego, związanego z instrukcjami SIMD, jak i do poziomu programistycznego, w którym operacje zapisywane są jako działania na całych tablicach danych.

W analizie danych GPX zapis tablicowy może być użyteczny dla operacji wykonywanych na sekwencjach wartości liczbowych, takich jak obliczanie różnic wysokości, wykonywanie funkcji trygonometrycznych dla wielu punktów lub sumowanie odległości między kolejnymi próbkami trasy. Tego typu obliczenia można często zapisać jako działania na tablicach numerycznych, ograniczając udział jawnych pętli wykonywanych bezpośrednio w kodzie Pythona.

Istotnym czynnikiem wpływającym na możliwość efektywnego wykorzystania takiego podejścia jest reprezentacja danych. Dane jednorodne, liczbowe i ułożone w sposób sprzyjający lokalności przestrzennej łatwiej zapisać jako operacje tablicowe niż dane rozproszone jako wiele odrębnych obiektów [5]. W języku Python dane programu są reprezentowane jako obiekty posiadające tożsamość, typ i wartość [6], dlatego porównanie list Pythona z jednorodnymi tablicami numerycznymi wymaga uwzględnienia nie tylko samego algorytmu, lecz także sposobu reprezentacji danych.

Przykładem jawnego wskazania możliwości wykonania pętli w sposób wektorowy jest konstrukcja `simd` dostępna w standardzie OpenMP. Służy ona do oznaczenia pętli, której iteracje mogą zostać wykonane z użyciem instrukcji wektorowych, jeżeli spełnione są warunki poprawnościowe dotyczące między innymi zależności między iteracjami [11]. Przykład ten pokazuje, że efektywna wektoryzacja wymaga odpowiedniej struktury obliczeń, a nie tylko samego faktu pracy na danych liczbowych.

Nie każda operacja zapisana jako działanie na tablicach prowadzi automatycznie do skrócenia czasu wykonania. Efektywność takiego podejścia zależy od charakteru algorytmu, układu danych w pamięci oraz sposobu wykonania operacji przez użyte narzędzia obliczeniowe.

1.2 Dane GPS i format GPX

Dane wykorzystywane w analizie tras sportowych można traktować jako uporządkowany zapis kolejnych położenia użytkownika. W formacie GPX ślad trasy jest reprezentowany przez element `trk`, który może zawierać segmenty `trkseg`. Segmenty te składają się z kolejnych punktów śladu `trkpt`, opisujących przebieg trasy [3]. Pojedynczy punkt śladu zawiera współrzędne geograficzne zapisane w atrybutach `lat` i `lon`, a dodatkowo może zawierać wysokość `ele` oraz czas pomiaru `time` [3].

Kolejność punktów jest istotna, ponieważ pozwala obliczać odległości między sąsiednimi pozycjami, wyznaczać zmiany wysokości oraz porównywać przebieg śladu z trasą referencyjną. W dalszych częściach sekcji omówiono charakter danych GPS, strukturę pliku GPX oraz znaczenie układów współrzędnych w analizie tras.

1.2.1 Charakterystyka danych GPS

Dane GPS rejestrowane podczas aktywności sportowej mają charakter sekwencyjny. Kolejne punkty zapisu odpowiadają następującym po sobie pomiarom położenia użytkownika, dlatego ślad trasy można analizować jako uporządkowany ciąg próbek. Taka reprezentacja jest podstawą operacji polegających na porównywaniu sąsiednich punktów, między innymi przy obliczaniu dystansu oraz zmian wysokości.

Pomiar GPS/GNSS może zawierać zakłócenia. Jednym z typowych źródeł błędów jest wielotorowość sygnału, czyli sytuacja, w której sygnał satelitarny dociera do odbiornika różnymi drogami wskutek odbić od przeszkód terenowych lub obiektów znajdujących się w otoczeniu [13]. Z tego powodu wyniki obliczeń wykonywanych na podstawie śladów GPS należy interpretować z uwzględnieniem ograniczonej dokładności danych wejściowych.

1.2.2 Struktura pliku GPX

GPX jest formatem zapisu danych geograficznych opartym na XML. Dokumentacja GPX 1.1 opisuje plik jako hierarchię elementów XML, w której elementem głównym jest `gpx`. [3] W analizie śladów sportowych szczególne znaczenie ma część struktury związana z zapisem trasy jako śladu, obejmująca elementy `trk`, `trkseg` oraz `trkpt`.

Wykorzystywana w dalszym przetwarzaniu hierarchia elementów ma następującą postać:

- `gpx` – element główny dokumentu,
- `metadata` – metadane pliku,
- `trk` – ślad zarejestrowanej aktywności,
- `trkseg` – segment śladu,
- `trkpt` – pojedynczy punkt śladu.

Punkt śladu `trkpt` zawiera atrybuty `lat` i `lon`, określające odpowiednio szerokość i długość geograficzną punktu. Dodatkowo punkt może zawierać element `ele`, przechowujący wysokość, oraz element `time`, określający czas rejestracji próbki [3]. Brak wysokości lub czasu nie musi oznaczać braku punktu geometrycznego, ponieważ położenie punktu wynika ze współrzędnych `lat` i `lon`. W przygotowanej implementacji przyjęto jednak założenie pracy na kompletnych punktach wejściowych, co zostanie omówione w rozdziale implementacyjnym.

Listing 1.1 przedstawia uproszczony fragment pliku GPX obejmujący elementy istotne dla dalszego przetwarzania: `metadata`, `trk`, `trkseg`, `trkpt`, atrybuty `lat` i `lon` oraz elementy `ele` i `time`.

Listing 1.1: Fragment przykładowego pliku GPX

```
1 <gpx creator="Garmin Connect" version="1.1"
2   xmlns="http://www.topografix.com/GPX/1/1">
3   <metadata>
4     <time>2026-03-12T18:26:20.000Z</time>
5   </metadata>
6   <trk>
```

```
7      <type>Przykładowy plik GPX</type>
8      <trkseg>
9          <trkpt lat="51.231467" lon="22.50682">
10              <ele>124.7</ele>
11              <time>2026-03-12T18:26:27.000Z</time>
12          </trkpt>
13          <trkpt lat="51.231454" lon="22.506844">
14              <ele>124</ele>
15              <time>2026-03-12T18:26:28.000Z</time>
16          </trkpt>
17          <trkpt lat="51.231439" lon="22.506866">
18              <ele>124</ele>
19              <time>2026-03-12T18:26:29.000Z</time>
20          </trkpt>
21      </trkseg>
22  </trk>
23 </gpx>
```

Przedstawiony fragment pokazuje sposób zapisu pojedynczego punktu śladu. Atrybuty `lat` i `lon` określają położenie punktu, element `ele` przechowuje wysokość, natomiast `time` zawiera czas rejestracji próbki. Na podstawie kolejnych elementów `trkpt` można utworzyć sekwencję punktów trasy stanowiącą dane wejściowe dla operacji analitycznych.

W plikach GPX mogą występować również rozszerzenia oraz dodatkowe informacje wykraczające poza podstawowe elementy punktów śladu.[3] W obliczeniach wykonywanych w przygotowanym programie wartości takie jak dystans czy suma przewyższeń nie są jednak odczytywane jako gotowe metryki z pliku, lecz wyznaczane ponownie na podstawie kolejnych punktów trasy. Pozwala to porównywać warianty implementacyjne wykonujące te same operacje na tych samych danych wejściowych.

Format GPX, jako format tekstowy oparty na XML, wymaga wydobycia danych ze struktury dokumentu przed rozpoczęciem właściwych obliczeń.[3] Dlatego w implementacji konieczne jest oddzielenie etapu parsowania pliku od etapu obliczeń wykonywanych na przygotowanej reprezentacji danych.

1.2.3 Układy współrzędnych w analizie tras

Zapis pozycji w danych GPS oraz w formacie GPX opiera się na globalnym układzie odniesienia WGS84. W tym układzie pozycja jest opisywana za pomocą współrzędnych geograficznych: szerokości i długości geograficznej. Dokumentacja GPX 1.1 wskazuje, że współrzędne zapisane w pliku GPX odnoszą się do układu WGS84, natomiast wartości takie jak wysokość są wyrażane w jednostkach metrycznych [14, 3].

Konsekwencją takiego sposobu zapisu jest konieczność rozróżnienia obliczeń wykonywanych na współrzędnych geograficznych i obliczeń wymagających układu metrycznego.

Szerokość i długość geograficzna są wyrażane w stopniach, dlatego nie powinny być traktowane bezpośrednio jako współrzędne kartezjańskie w metrach. Przy obliczaniu odległości między punktami na powierzchni Ziemi można zastosować wzór uwzględniający geograficzny charakter współrzędnych, natomiast operacje geometryczne wykonywane w płaszczyźnie wymagają reprezentacji, w której jednostką odległości jest metr.

Przekształcenie współrzędnych geograficznych na płaszczyznę wymaga zastosowania odwzorowania kartograficznego. Ponieważ każde odwzorowanie wiąże się z określonym rodzajem i rozkładem zniekształceń, dobór układu płaskiego powinien być związany z obszarem opracowania oraz charakterem wykonywanych analiz.[15] Nie istnieje więc jeden uniwersalny układ płaski, który byłby jednakowo odpowiedni dla wszystkich obszarów i wszystkich zastosowań.

W implementacji przyjęto, że analizowane trasy znajdują się na obszarze Polski. Dla operacji geometrycznych wykonywanych w płaszczyźnie, takich jak porównywanie punktu śladu z segmentem trasy referencyjnej, zastosowano układ współrzędnych płaskich PL-1992, oznaczany kodem EPSG:2180. Układ ten jest opisany jako dwuwymiarowy układ kartezjański z jednostką metra, co pozwala interpretować wyniki takich obliczeń w jednostkach metrycznych [16].

1.3 Metody analizy i porównywania tras

Analiza tras sportowych obejmuje zarówno wyznaczanie podstawowych parametrów pojedynczego śladu, jak i ocenę relacji między śladem zawodnika a trasą referencyjną. W praktyce użytkowej, na przykład w systemach do planowania i analizy aktywności, często automatycznie prezentowane są takie informacje jak długość trasy, profil wysokościowy, suma podejść i zejść czy szacowany czas przejazdu.[1, 2] Są to metryki opisujące pojedynczą trasę lub pojedynczy zapis aktywności.

W przypadku zawodów sportowych istotne może być jednak nie tylko opisanie jednej trasy, lecz także sprawdzenie zgodności rzeczywistego śladu zawodnika z przebiegiem trasy referencyjnej. Takie porównanie pozwala ocenić, czy zawodnik poruszał się w pobliżu wyznaczonej trasy, gdzie wystąpiły największe odchylenia oraz jaki udział punktów śladu mieścił się w przyjętych progach tolerancji. Wymaga to zastosowania innych operacji niż samo obliczenie dystansu lub przewyższeń, ponieważ konieczne jest analizowanie relacji pomiędzy punktami śladu a geometrią trasy wzorcowej.

Metody wykorzystywane do analizy tras GPX można więc podzielić według rodzaju danych, na których operują, oraz według kosztu obliczeniowego. Najprostsze operacje dotyczą pojedynczego śladu i polegają na przejściu po kolejnych punktach trasy. Do tej grupy należą obliczanie dystansu oraz wyznaczanie sumy podejść i zejść. Ich wynik zależy od relacji między sąsiednimi próbkami, dlatego naturalnym modelem obliczeń jest

sekwencyjne przetwarzanie uporządkowanego ciągu punktów.

Bardziej złożone są metody porównujące ślad zawodnika z trasą referencyjną. Wymagają one analizy relacji między dwoma przebiegami trasy, na przykład przez wyznaczanie odległości punktów śladu od segmentów linii referencyjnej albo przez porównanie dwóch sekwencji punktów jako całości. W tym drugim przypadku wykorzystana może być dyskretna odległość Frécheta, która uwzględnia nie tylko położenie punktów, lecz także ich kolejność w obu porównywanych sekwencjach.[22]

Podział metod ma znaczenie również dla sposobu implementacji. Operacje wykonywane na jednej sekwencji danych, takie jak obliczanie różnic wysokości albo odległości między sąsiednimi punktami, łatwiej zapisać jako działania na tablicach numerycznych. Wynika to z faktu, że kolejne wartości mogą być przetwarzane według tego samego schematu obliczeń. Metody porównujące dwa przebiegi trasy, zwłaszcza oparte na relacjach punkt-segment lub programowaniu dynamicznym, mają bardziej złożoną strukturę obliczeń i nie zawsze poddają się prostej wektoryzacji.

1.3.1 Obliczanie dystansu i przewyższeń

Jedną z podstawowych operacji wykonywanych na śladzie GPX jest obliczenie długości trasy. Ponieważ ślad ma postać uporządkowanej sekwencji punktów, dystans można wyznaczyć jako sumę odległości między kolejnymi punktami. W badaniach dotyczących tras trailowych i górskich odległość między sąsiednimi punktami pliku GPX obliczano na podstawie współrzędnych geograficznych, traktując ją jako odległość poziomą, natomiast pionowe przemieszczenie między punktami analizowano oddzielnie jako składnik sumy podejść lub zejść.[17]

Odległość między kolejnymi punktami może być obliczana z użyciem wzoru Haversine’a, który pozwala wyznaczyć przybliżoną odległość po wielkim kole między dwoma punktami opisanymi szerokością i długością geograficzną [18]. Wzór ten zakłada model sferyczny Ziemi, dlatego wynik należy traktować jako przybliżenie wystarczające dla przyjętego zakresu analiz tras GPX. Dla punktów o szerokościach geograficznych ϕ_1 , ϕ_2 oraz długościach geograficznych λ_1 , λ_2 , przyjmuje się:

$$\Delta\phi = \phi_2 - \phi_1,$$

$$\Delta\lambda = \lambda_2 - \lambda_1.$$

Następnie obliczana jest wartość pomocnicza:

$$a = \sin^2\left(\frac{\Delta\phi}{2}\right) + \cos\phi_1 \cos\phi_2 \sin^2\left(\frac{\Delta\lambda}{2}\right).$$

Odległość między punktami jest wyznaczana jako:

$$d = 2R \operatorname{atan2}(\sqrt{a}, \sqrt{1-a}),$$

gdzie R oznacza przyjęty promień Ziemi, a d oznacza odległość między punktami po powierzchni sfery. Przed podstawieniem do wzoru wartości szerokości i długości geograficznej muszą zostać wyrażone w radianach.[18]

Długość całej trasy obliczana jest jako suma odległości dla wszystkich par sąsiednich punktów śladu:

$$D = \sum_{i=1}^{n-1} d(p_i, p_{i+1}),$$

gdzie n oznacza liczbę punktów trasy, a $d(p_i, p_{i+1})$ oznacza odległość między punktem p_i oraz punktem p_{i+1} .

Suma podejść i suma zejść są wyznaczane na podstawie różnic wysokości między kolejnymi punktami. Dla dwóch sąsiednich punktów różnicę wysokości można zapisać jako:

$$\Delta h_i = h_{i+1} - h_i.$$

Suma podejść obejmuje dodatnie różnice wysokości:

$$H^+ = \sum_{i=1}^{n-1} \max(\Delta h_i, 0),$$

natomiast suma zejść obejmuje wartości bezwzględne różnic ujemnych:

$$H^- = \sum_{i=1}^{n-1} \max(-\Delta h_i, 0).$$

Wartości dystansu oraz przewyższeń zależą od jakości i rozdzielczości danych wejściowych. W badaniach nad trasami trailowymi wykazano, że zmiana rozdzielczości przestrzennej pliku GPX wpływa na obliczany dystans oraz skumulowane przewyższenia.[17] Dodatkowo błąd pomiaru GPS może prowadzić do zniekształcenia długości trajektorii, w tym do jej systematycznego zawyżania w określonych warunkach.[19]

Z punktu widzenia kosztu obliczeniowego obliczanie dystansu, sumy podejść i sumy zejść wymaga jednokrotnego przejścia po sekwencji punktów. Każdy punkt, poza pierwszym, jest porównywany z punktem poprzednim, dlatego liczba operacji rośnie liniowo wraz z liczbą punktów trasy. Struktura tych obliczeń sprzyja zapisowi w postaci operacji tablicowych. Różnice wysokości oraz różnice współrzędnych między sąsiednimi punktami mogą być wyznaczane dla wielu elementów sekwencji według tego samego schematu obliczeń. Nie oznacza to automatycznie skrócenia czasu wykonania w każdym przypadku, ale sprawia, że obliczanie dystansu oraz przewyższeń jest naturalnym przykładem operacji podatnych na wektoryzację.

1.3.2 Porównywanie trasy z trasą referencyjną

Porównywanie śladu zawodnika z trasą referencyjną wymaga przyjęcia geometrycznej reprezentacji trasy wzorcowej. W standardzie OGC Simple Feature Access geometria typu `LineString` jest opisywana jako krzywa z interpolacją liniową między punktami, a każda kolejna para punktów definiuje segment linii [20]. W pracy przyjęto analogiczny model: trasa referencyjna jest traktowana jako linia łamana utworzona z uporządkowanej sekwencji punktów.

Sąsiednie punkty trasy referencyjnej tworzą segmenty, względem których oceniane jest położenie punktów śladu zawodnika. Lokalne porównanie przebiegu tras można wówczas sprowadzić do obliczenia odległości punktu od segmentu trasy referencyjnej. Dla każdego punktu śladu zawodnika można wyznaczyć minimalną odległość do jednego z segmentów trasy wzorcowej, a następnie traktować ją jako lokalne odchylenie od przebiegu trasy referencyjnej.

Na podstawie zbioru takich lokalnych odległości można obliczać miary podsumowujące, takie jak średnie odchylenie, największe odchylenie oraz udział punktów mieszczących się w przyjętych progach tolerancji. Takie podejście jest zgodne z implementacją programu, w której dla każdego punktu śladu zawodnika obliczana jest minimalna odległość od segmentów trasy referencyjnej, a następnie wyznaczane są wartości zbiorcze.

Dla odcinka AB oraz punktu C najbliższy punkt na odcinku można wyznaczyć przez rzutowanie punktu C na prostą przechodzącą przez A i B , a następnie sprawdzenie, czy punkt rzutu znajduje się wewnątrz odcinka. Jeżeli rzut znajduje się na odcinku, to on jest punktem najbliższym względem C . Jeżeli rzut wypada poza odcinkiem, najbliższym punktem jest jeden z końców odcinka [21].

Niech A , B oraz C oznaczają punkty w układzie płaskim. Punkt na prostej przechodzącej przez końce odcinka można zapisać parametrycznie jako:

$$P(t) = A + t(B - A).$$

Wartość parametru t można obliczyć z wykorzystaniem iloczynu skalarnego:

$$t = \frac{(C - A) \cdot (B - A)}{\|B - A\|^2}.$$

Parametr t określa położenie rzutu punktu C względem odcinka AB . Jeżeli mianownik $\|B - A\|^2$ jest równy zero, segment ma zerową długość i odległość można sprowadzić do odległości punktu C od punktu A . W przeciwnym przypadku wyróżnia się trzy sytuacje:

- dla $t < 0$ najbliższym punktem odcinka jest punkt A ,
- dla $0 \leq t \leq 1$ najbliższym punktem jest rzut $P(t)$,
- dla $t > 1$ najbliższym punktem odcinka jest punkt B .

Odległość punktu śladu od segmentu może zostać obliczona jako odległość euklidesowa między punktem C a wyznaczonym punktem najbliższym. Jeżeli punkt śladu jest porównywany z wieloma segmentami trasy referencyjnej, minimalna z otrzymanych odległości może być traktowana jako lokalne odchylenie śladu od przebiegu trasy wzorcowej.

Przy prostym podejściu obliczeniowym każdy punkt śladu zawodnika jest porównywany ze wszystkimi segmentami trasy referencyjnej. Złożoność obliczeniowa takiej metody zależy więc od liczby punktów śladu oraz liczby segmentów trasy referencyjnej. Dla m punktów śladu zawodnika i s segmentów trasy referencyjnej konieczne jest sprawdzenie $m \cdot s$ relacji punkt–segment, dlatego złożoność czasowa metody wynosi $O(m \cdot s)$.

1.3.3 Dyskretna odległość Fréchet

Dyskretna odległość Fréchet jest miarą podobieństwa krzywych łamanych reprezentowanych przez skończone sekwencje punktów. W odróżnieniu od lokalnego porównywania punktów z segmentami trasy referencyjnej, metoda ta uwzględnia uporządkowanie punktów na obu porównywanych krzywych [22]. Cecha ta jest istotna przy analizie śladów GPX, ponieważ trasa ma postać uporządkowanej sekwencji położeń, a nie nieuporządkowanego zbioru punktów.

Eiter i Mannila definiują dyskretną odległość Fréchet dla dwóch krzywych łamanych P oraz Q , reprezentowanych przez sekwencje punktów:

$$\sigma(P) = (u_1, \dots, u_p),$$

$$\sigma(Q) = (v_1, \dots, v_q).$$

Podstawą definicji jest uporządkowane parowanie punktów obu sekwencji, określane jako *coupling* [22]. Parowanie rozpoczyna się od pierwszych punktów obu sekwencji i kończy na ich ostatnich punktach. W kolejnych krokach można przejść do następnego punktu pierwszej krzywej, do następnego punktu drugiej krzywej albo do następnych punktów obu krzywych jednocześnie. Dzięki temu zachowana zostaje kolejność punktów, a jednocześnie możliwe jest porównywanie sekwencji o różnej liczbie próbek.

Długość danego parowania jest określana jako największa odległość pomiędzy punktami występującymi w sparowanych parach. Dyskretna odległość Fréchet odpowiada najmniejszej możliwej wartości tej największej odległości, rozpatrywanej po wszystkich dopuszczalnych uporządkowanych parowaniach punktów obu krzywych [22]. Wynik tej miary nie zależy więc od pojedynczego lokalnego dopasowania, lecz od najlepszego możliwego przejścia po obu sekwencjach z zachowaniem ich kolejności.

Obliczenie dyskretnej odległości Fréchet można przeprowadzić metodą programowania dynamicznego. Dla każdej pary punktów u_i oraz v_j wyznaczana jest wartość opisująca najlepsze dopasowanie początkowych fragmentów obu sekwencji kończących się odpowied-

nio w tych punktach. Wartość ta zależy od wcześniej obliczonych sąsiednich stanów oraz od odległości $d(u_i, v_j)$ między aktualnie porównywanymi punktami [22].

Przy indeksowaniu od 1 warunki brzegowe można zapisać następująco:

$$\begin{aligned} c(1, 1) &= d(u_1, v_1), \\ c(i, 1) &= \max\{c(i-1, 1), d(u_i, v_1)\}, \\ c(1, j) &= \max\{c(1, j-1), d(u_1, v_j)\}. \end{aligned}$$

Dla przypadku $i > 1$ oraz $j > 1$ zależność rekurencyjną można zapisać następująco:

$$c(i, j) = \max(\min\{c(i-1, j), c(i-1, j-1), c(i, j-1)\}, d(u_i, v_j)).$$

Wartość $c(i, j)$ oznacza najmniejszą możliwą wartość maksymalnej odległości potrzebnej do dopasowania początkowych fragmentów obu sekwencji kończących się w punktach u_i oraz v_j . Końcową wartością dyskretnej odległości Fréchet'a jest $c(p, q)$, czyli wartość odpowiadająca dopasowaniu pełnych sekwencji punktów.

Dla sekwencji o długościach p oraz q algorytm wymaga rozpatrzenia par punktów obu krzywych, dlatego jego złożoność czasowa wynosi $O(pq)$ [22]. Jeżeli implementacja przechowuje pełną macierz programowania dynamicznego, złożoność pamięciowa również wynosi $O(pq)$. Ma to znaczenie praktyczne przy porównywaniu długich tras GPX, ponieważ liczba komórek macierzy rośnie wraz z iloczynem liczby punktów obu śladów.

W analizie śladów GPX dyskretna odległość Fréchet'a może służyć do porównania trasy zawodnika z trasą referencyjną na poziomie całego przebiegu trasy. Jej koszt obliczeniowy jest jednak większy niż w przypadku operacji liniowych, ponieważ wymaga rozpatrzenia relacji między punktami obu porównywanych sekwencji.

Rozdział 2

Reprezentacja danych i narzędzia obliczeniowe w Pythonie

Rozdział przedstawia zagadnienia związane z reprezentacją danych liczbowych oraz sposobem wykonywania obliczeń w języku Python. Są one istotne z punktu widzenia implementacji metod analizy tras GPX, ponieważ te same metryki mogą zostać obliczone przy użyciu różnych struktur danych i różnych mechanizmów wykonania kodu. Różnice między wariantami implementacyjnymi nie dotyczą definicji obliczanych miar, lecz sposobu przechowywania danych, organizacji pętli oraz wykorzystania bibliotek numerycznych.

W dalszej części rozdziału omówiono trzy podejścia zastosowane w projekcie: wariant oparty na wbudowanych strukturach języka Python, wariant wykorzystujący tablice NumPy oraz wariant kompilowany z użyciem biblioteki Numba. Wariant listowy pełni rolę punktu odniesienia, ponieważ pozwala zapisać algorytmy w sposób bezpośredni i czytelny. NumPy umożliwia przechowywanie danych w jednorodnych tablicach numerycznych oraz zapis części operacji w postaci działań tablicowych. Numba pozwala natomiast kompilować wybrane funkcje numeryczne, o ile ich struktura oraz użyte typy danych są zgodne z ograniczeniami kompilatora JIT. Ma to znaczenie zwłaszcza dla metod, w których zachowanie jawnych pętli jest prostsze lub bardziej naturalne niż próba pełnej wektoryzacji.

2.1 Python i reprezentacja danych liczbowych

Po odczytaniu pliku GPX dane trasy nie są już przetwarzane jako dokument XML, lecz jako sekwencje wartości liczbowych wykorzystywane w dalszych operacjach analitycznych. Do najważniejszych wartości wykorzystywanych w dalszych obliczeniach należą szerokości geograficzne, długości geograficzne oraz wysokości. Znaczniki czasu mogą występować w plikach GPX, jednak w aktualnym zakresie implementacji nie są wykorzystywane przez funkcje analityczne, dlatego nie są przechowywane w strukturach zwracanych przez parser.

W przygotowanej implementacji dane te są po parsowaniu przechowywane jako osobne sekwencje składowych punktu, a nie jako pojedyncza lista złożonych obiektów punktów.

Język Python umożliwia zapis algorytmów w sposób czytelny i bezpośredni, jednak jego ogólny model danych różni się od modelu jednorodnych tablic liczbowych wykorzystywanych w obliczeniach numerycznych. Zgodnie z dokumentacją języka Python wszystkie dane w programie są reprezentowane przez obiekty lub relacje między obiektami, a każdy obiekt posiada tożsamość, typ i wartość [6]. Ma to znaczenie przy przetwarzaniu dużych sekwencji liczb, ponieważ operacje wykonywane na obiektach Pythona mogą wiązać się z innym narzutem wykonania niż operacje wykonywane na jednorodnych tablicach numerycznych [23].

W analizie tras GPX szczególnie istotne są operacje powtarzane dla wielu kolejnych punktów śladu. Obliczanie dystansu wymaga przechodzenia po parach sąsiednich punktów, suma podejść i zejść wynika z różnic wysokości między kolejnymi próbkami, natomiast porównywanie śladu zawodnika z trasą referencyjną wymaga analizy relacji między punktami jednej trasy a segmentami drugiej trasy. Są to operacje o różnej strukturze obliczeniowej, dlatego nie wszystkie w takim samym stopniu nadają się do zapisu w postaci działań tablicowych.

Z tego powodu w projekcie zastosowano trzy warianty implementacyjne. Wariant bazowy wykorzystuje listy Pythona i stanowi punkt odniesienia dla pozostałych rozwiązań. Wariant NumPy przechowuje dane w jednorodnych tablicach numerycznych i pozwala zapisać część obliczeń jako działania na całych sekwencjach wartości. Wariant Numba korzysta z tablic NumPy jako danych wejściowych, ale kluczowe pętle numeryczne są wykonywane w funkcjach kompilowanych z użyciem mechanizmu JIT. Porównanie tych podejść pozwala ocenić wpływ reprezentacji danych i mechanizmu wykonania kodu na czas działania tych samych metod analizy tras GPX.

2.1.1 Listy Pythona jako wariant bazowy

Jednym z podstawowych sposobów reprezentowania sekwencji danych w języku Python jest lista. W kontekście analizy tras GPX jest to rozwiązanie naturalne z punktu widzenia czytelności kodu, ponieważ ślad trasy również ma postać uporządkowanej sekwencji pomiarów. W przygotowanej implementacji dane odczytane z pliku GPX nie są jednak przechowywane jako jedna lista złożonych obiektów punktów, lecz jako osobne listy odpowiadające poszczególnym składowym śladu: szerokościom geograficznym, długościom geograficznym oraz wysokościami. Taki układ ułatwia bezpośrednie przechodzenie po wybranych wartościach potrzebnych w danej operacji.

Wariant listowy należy rozpatrywać w kontekście obiektowego modelu danych Pythona. Zgodnie z dokumentacją języka każdy obiekt posiada tożsamość, typ oraz wartość [6]. W praktyce oznacza to, że wartości liczbowe znajdujące się na liście są obsługiwane jako

obiekty Pythona, a nie jako elementy jednorodnego bufora liczbowego porównywalnego z tablicą typu `float64`.

W standardowej implementacji CPython lista jest strukturą zmiennej długości opartą na tablicy referencji do obiektów, a nie na tablicy przechowującej bezpośrednio wartości liczbowe [7]. Dzięki temu lista jest elastyczna i pozwala łatwo dodawać kolejne elementy, na przykład podczas parsowania punktów śladu GPX. Jednocześnie taka reprezentacja różni się od jednorodnych tablic numerycznych, w których elementy mają ustalony typ i są przeznaczone bezpośrednio do obliczeń na danych liczbowych.

Wariant listowy pełni w przygotowanej implementacji rolę wariantu bazowego. Jego zaletą jest prostota zapisu oraz bezpośrednie odwzorowanie algorytmów opisanych w rozdziale teoretycznym. Obliczanie dystansu można w takim wariacie zrealizować przez przechodzenie po parach sąsiednich wartości szerokości i długości geograficznej. Suma podejść i zejść może być wyznaczana przez porównywanie kolejnych wartości wysokości, natomiast porównywanie śladu z trasą referencyjną przez jawne sprawdzanie relacji między punktami i segmentami.

W części geometrycznej programu, po przekształceniu współrzędnych do układu metrycznego, wariant listowy korzysta z wbudowanych struktur Pythona. W części geometrycznej programu, po przekształceniu współrzędnych do układu metrycznego, wariant listowy nadal korzysta z wbudowanych struktur Pythona. Istotne jest tutaj przede wszystkim zachowanie bezpośredniego, sekwencyjnego sposobu zapisu algorytmu. Wariant listowy różni się więc od wariantu NumPy nie definicją wykonywanych obliczeń, lecz sposobem organizacji danych i realizacji pętli.

Wariant listowy jest więc czytelny i elastyczny, ale nie tworzy jednorodnego bufora wartości liczbowych porównywalnego z tablicą NumPy typu `float64`. Dzięki temu może pełnić rolę punktu odniesienia dla wariantów wykorzystujących tablice NumPy oraz kompilację JIT.

2.1.2 NumPy i tablice numeryczne

NumPy jest jedną z podstawowych bibliotek wykorzystywanych w języku Python do obliczeń numerycznych i programowania tablicowego. Jej centralną strukturą danych jest `ndarray`, czyli wielowymiarowa tablica elementów tego samego typu i rozmiaru. Typ elementów przechowywanych w tablicy jest określany przez obiekt `dtype` związany z daną tablicą [25]. Taki model różni się od wbudowanych list Pythona, które przechowują odwołania do obiektów i mogą zawierać elementy różnych typów.

Znaczenie NumPy w obliczeniach naukowych wynika z możliwości organizowania danych w postaci tablic oraz wykonywania wielu operacji matematycznych bez konieczności zapisywania ich jako jawnych pętli bezpośrednio w kodzie Pythona. Harris i in. opisują NumPy jako podstawową bibliotekę programowania tablicowego w ekosystemie naukowego

Pythona [24]. W kontekście niniejszej pracy ma to znaczenie, ponieważ dane odczytane z plików GPX mają charakter liczbowy i sekwencyjny: obejmują między innymi szerokości geograficzne, długości geograficzne oraz wysokości punktów śladu.

W przygotowanej implementacji wariant NumPy wykorzystuje osobne tablice dla wybranych składowych śladu, takich jak współrzędne i wysokości. Taki układ danych pozwala wykonywać operacje na całych sekwencjach jednej składowej, na przykład na wszystkich wysokościach albo na wszystkich szerokościach geograficznych.

W wariacie NumPy wykorzystano między innymi funkcje uniwersalne, określane jako **ufuncs**. Są to funkcje działające element po elemencie na tablicach **ndarray** i wspierające mechanizmy takie jak broadcasting [26]. W kontekście analizy tras pozwala to zapisać wybrane działania, na przykład funkcje trygonometryczne albo odejmowanie sąsiednich wartości, jako operacje na całych tablicach.

Podobnie obliczanie sumy podejść i zejść można zapisać jako operację na przesuniętych fragmentach tablicy wysokości. Różnice wysokości są obliczane dla całej sekwencji, a następnie filtrowane przy użyciu masek logicznych. Dodatnie różnice składają się na sumę podejść, natomiast wartości ujemne, po zmianie znaku lub użyciu wartości bezwzględnej, tworzą sumę zejść. Ten typ operacji dobrze odpowiada modelowi tablicowemu, ponieważ ta sama operacja jest stosowana do wielu kolejnych elementów danych.

Należy jednak odróżnić wykorzystanie tablic NumPy od pełnej wektoryzacji całego algorytmu. NumPy umożliwia zapis wielu operacji jako działań tablicowych, lecz nie każda metoda analizy tras GPX ma strukturę pozwalającą na całkowite usunięcie pętli zapisanych w Pythonie. W przypadku porównywania śladu zawodnika z trasą referencyjną obliczenie odległości jednego punktu od wszystkich segmentów trasy referencyjnej może zostać zapisane jako operacja na tablicach. Jednocześnie funkcja nadal przechodzi po kolejnych punktach śladu zawodnika, dlatego jest to przykład częściowej, a nie pełnej wektoryzacji.

Nie wszystkie metody można jednak zapisać jako pełne operacje tablicowe. Algorytmy zależne od wcześniej obliczonych stanów, takie jak metody programowania dynamicznego, mogą nadal wymagać pętli iteracyjnych. W takich przypadkach NumPy może służyć głównie do reprezentacji danych, a nie do całkowitego usunięcia pętli z algorytmu.

Mechanizm broadcastingu pozwala wykonywać operacje na tablicach o różnych, lecz zgodnych kształtach. Dokumentacja NumPy wskazuje, że w takim przypadku pętle mogą być wykonywane po stronie kodu niskopoziomowego zamiast bezpośrednio w Pythonie, bez niepotrzebnego kopiowania danych [27]. W tej pracy większe znaczenie mają jednak proste operacje na tablicach jednowymiarowych, wycinki tablic oraz maski logiczne.

Wariant NumPy pełni więc w pracy rolę reprezentacji tablicowej i narzędzia do zapisu tych obliczeń, które naturalnie można przedstawić jako operacje na jednorodnych sekwencjach wartości liczbowych. Porównanie tego wariantu z podejściem listowym i z

wariantem Numba pozwala sprawdzić, jak zapis tablicowy wpływa na czas wykonania metod o różnej strukturze obliczeniowej.

2.1.3 Numba i kompilacja JIT

Trzecim wariantem wykorzystanym w implementacji operacji analizy tras GPX jest wariant oparty na kompilacji JIT, czyli *just-in-time compilation*. Podejście to różni się zarówno od wariantu listowego, w którym pętle wykonywane są bezpośrednio przez interpreter Pythona, jak i od wariantu NumPy, w którym wybrane obliczenia są zapisywane jako operacje tablicowe. W przypadku kompilacji JIT wybrane funkcje są tłumaczone do postaci wykonywalnej w czasie działania programu, a następnie mogą być uruchamiane dla określonych typów argumentów.

W projekcie zastosowano bibliotekę Numba, która została zaprojektowana jako kompilator JIT dla Pythona ukierunkowany na obliczenia naukowe oraz tablicowe [28]. Numba może być szczególnie użyteczna w sytuacjach, w których algorytm ma charakter numeryczny, operuje na typach liczbowych lub tablicach NumPy, a jego logika jest naturalnie zapisana za pomocą jawnych pętli. Nie oznacza to jednak, że każda funkcja Pythona może zostać automatycznie przyspieszona. Możliwość skutecznej kompilacji zależy od użytych typów danych, instrukcji oraz bibliotek zewnętrznych.

W dokumentacji Numby wyróżniane są dwa podstawowe tryby kompilacji: *nopython mode* oraz *object mode*. Tryb *nopython* zakłada wykonanie skompilowanej funkcji bez operowania na ogólnych obiektach Pythona w głównej części obliczeń, natomiast tryb *object mode* zachowuje obsługę obiektów Pythona i zwykle ogranicza potencjalne korzyści wydajnościowe [29]. Z tego powodu w obliczeniach numerycznych istotne jest takie zapisanie funkcji, aby mogła zostać skompilowana w trybie *nopython*. W implementacji programu wykorzystano w tym celu dekorator `@njit`, będący wygodną formą wymuszenia kompilacji w tym trybie.

Wariant Numba zastosowano przede wszystkim dla metod, których struktura obliczeń utrudnia pełne przekształcenie do postaci operacji tablicowych NumPy. Dotyczy to między innymi porównywania punktów śladu zawodnika z segmentami trasy referencyjnej oraz obliczania dyskretnej odległości Fréchet’a. W obu przypadkach występują jawne pętle i zależności między kolejnymi etapami obliczeń. Wariant Numba pozwala zachować czytelną strukturę algorytmu opartą na pętlach, a jednocześnie może ograniczyć narzut ich wykonywania przez interpreter Pythona.

Kompilacja JIT nie musi obejmować całej funkcji wysokiego poziomu. W metodach wymagających przekształcenia współrzędnych do układu metrycznego część przygotowawcza, taka jak transformacja współrzędnych z użyciem zewnętrznej biblioteki, może pozostać poza zakresem kompilacji. Kompilowany jest wówczas przede wszystkim numeryczny rdzeń algorytmu, na przykład pętla obliczająca odległości punktów od segmentów albo pętla

wypełniająca macierz programowania dynamicznego.

Przy pomiarze czasu wykonania funkcji kompilowanych JIT należy uwzględnić koszt pierwszego wywołania funkcji. Pierwsze uruchomienie może obejmować kompilację dla konkretnych typów argumentów, dlatego nie powinno być bezpośrednio porównywane z kolejnymi wywołaniami funkcji już skompilowanej. Z tego powodu w eksperymentach wydajnościowych konieczne jest rozdzielenie etapu rozgrzewki lub wymuszenia kompilacji od właściwego pomiaru czasu wykonania.

Wariant Numba stanowi trzeci sposób wykonania tych samych metod analitycznych. Jego uwzględnienie pozwala porównać zapis tablicowy NumPy z podejściem opartym na kompilacji pętli numerycznych.

Rozdział 3

Projekt i implementacja oprogramowania do analizy tras GPX

Celem rozdziału jest przedstawienie sposobu realizacji programu służącego do analizy tras zapisanych w formacie GPX. Opis obejmuje strukturę projektu, parser plików GPX, warianty funkcji analitycznych oraz skrypty benchmarkowe. W projekcie przygotowano trzy warianty obliczeniowe: wariant bazowy oparty na listach Pythona, wariant wykorzystujący tablice NumPy oraz wariant kompilowany z użyciem biblioteki Numba.

Zakres implementacji obejmuje odczyt punktów śladu GPX, obliczanie dystansu, wyznaczanie sumy podejść i zejść, porównywanie śladu zawodnika z trasą referencyjną oraz obliczanie dyskretnej odległości Frécheta. Dodatkowo przygotowano skrypt generujący syntetyczne pliki GPX o kontrolowanej liczbie punktów, wykorzystywane później w benchmarkach.

3.1 Struktura projektu

Projekt został podzielony na katalogi odpowiadające kolejnym etapom pracy z danymi: przygotowaniu plików wejściowych, implementacji funkcji analitycznych oraz zapisowi wyników benchmarków. Taki podział pozwala oddzielić dane wejściowe od kodu programu oraz od wyników pomiarów czasu wykonania.

Listing 3.1: Ogólna struktura katalogów projektu

```
gpx_project/  
|-- data/  
|-- results/  
|   |-- benchmark_distance.json  
|   |-- benchmark_elevation.json  
|   |-- benchmark_route_comparison.json  
|   |-- benchmark_frechet_distance.json  
|-- scripts/
```

```
| |-- generate_gpx_data.py
| |-- gpx_parser.py
| |-- gpx_list.py
| |-- gpx_numpy.py
| |-- gpx_numba.py
| |-- gpx_benchmark_distance.py
| |-- gpx_benchmark_elevation.py
| |-- gpx_benchmark_route_comparison.py
| |-- gpx_benchmark_frechet_distance.py
|-- requirements.txt
```

Katalog `data` jest tworzony przez skrypt generujący dane testowe. W archiwum projektu nie musi zawierać pełnego zestawu plików GPX, ponieważ pliki te mają dużą objętość i mogą zostać odtworzone przez uruchomienie skryptu `generate_gpx_data.py`. Katalog `results` zawiera natomiast zapisane wyniki benchmarków wykorzystane w rozdziale poświęconym analizie czasu wykonania.

Katalog `data` zawiera pliki GPX wykorzystywane jako dane wejściowe. Są to syntetycznie wygenerowane trasy referencyjne oraz odpowiadające im trasy zawodnika. Nazwy plików mają postać:

`trasa_N.gpx`

`trasa_N_zawodnik.gpx`

gdzie `N` oznacza liczbę punktów śladu. Przykładowo para plików `trasa_10000.gpx` oraz `trasa_10000_zawodnik.gpx` zawiera odpowiednio trasę referencyjną i trasę zawodnika złożone z 10 000 punktów. Taki schemat nazewnictwa ułatwia automatyczne uruchamianie benchmarków dla różnych rozmiarów danych.

Katalog `scripts` zawiera kod źródłowy programu. W jego skład wchodzi moduły odpowiedzialne za generowanie danych, parsowanie plików GPX, implementację metod analitycznych oraz uruchamianie benchmarków. Najważniejsze pliki znajdujące się w tym katalogu to:

- `generate_gpx_data.py` – skrypt generujący syntetyczne pliki GPX,
- `gpx_parser.py` – moduł odpowiedzialny za odczyt danych z plików GPX,
- `gpx_list.py` – moduł zawierający wariant bazowy funkcji analitycznych,
- `gpx_numpy.py` – moduł zawierający wariant funkcji wykorzystujący tablice NumPy,
- `gpx_numba.py` – moduł zawierający wariant funkcji kompilowany z użyciem biblioteki Numba,
- `gpx_benchmark_distance.py` – skrypt mierzący czas obliczania dystansu,

- `gpx_benchmark_elevation.py` – skrypt mierzący czas obliczania przewyższeń,
- `gpx_benchmark_route_comparison.py` – skrypt mierzący czas porównywania trasy zawodnika z trasą referencyjną,
- `gpx_benchmark_frechet_distance.py` – skrypt mierzący czas obliczania dyskretnej odległości Frécheta.

Skrypt `generate_gpx_data.py` odpowiada za przygotowanie par plików GPX o ustalonej liczbie punktów. Dla każdej wartości N tworzony jest plik `trasa_N.gpx` oraz odpowiadający mu plik `trasa_N_zawodnik.gpx`. Pierwszy z nich pełni rolę trasy referencyjnej, natomiast drugi zawiera niewielkie odchylenia współrzędnych, dzięki czemu może być użyty do testowania metod porównujących dwie trasy.

Katalog `results` służy do przechowywania wyników benchmarków zapisanych w formacie JSON. Każda grupa operacji posiada osobny plik wynikowy, na przykład `benchmark_distance.json`, `benchmark_elevation.json`, `benchmark_route_comparison.json` oraz `benchmark_frechet_distance.json`. Oddzielenie wyników według rodzaju operacji ułatwia późniejszą analizę metod o różnej złożoności obliczeniowej.

Plik `requirements.txt` zawiera listę bibliotek wymaganych do uruchomienia projektu. W projekcie wykorzystywane są między innymi biblioteki NumPy, Numba oraz pyproj. NumPy służy do reprezentacji danych w postaci tablic numerycznych, Numba do kompilacji wybranych funkcji numerycznych, natomiast pyproj do przekształcania współrzędnych geograficznych do układu metrycznego w metodach wymagających obliczeń geometrycznych w płaszczyźnie.

Przepływ danych w projekcie można opisać następująco. Najpierw skrypt `generate_gpx_data.py` tworzy syntetyczne pliki GPX i zapisuje je w katalogu `data`. Następnie moduł `gpx_parser.py` odczytuje z tych plików współrzędne oraz wysokości punktów śladu. Tak przygotowane dane są przekazywane do funkcji analitycznych zaimplementowanych w trzech wariantach: listowym, NumPy oraz Numba. Ostatnim etapem jest uruchomienie skryptów benchmarkowych, które wykonują pomiary czasu i zapisują wyniki do katalogu `results`.

3.2 Implementacja parsera plików GPX

Pierwszym etapem przetwarzania danych jest odczyt pliku GPX i przekształcenie jego struktury XML do postaci struktur danych wykorzystywanych przez funkcje analityczne. W projekcie zaimplementowano dwie funkcje parsujące: wariant oparty na listach Pythona oraz wariant wykorzystujący tablice NumPy. Obie funkcje realizują tę samą logikę odczytu punktów śladu, lecz różnią się sposobem przechowywania wyników.

Parser został przygotowany dla plików GPX 1.1, w których punkty śladu są zapisane jako elementy `trkpt` w przestrzeni nazw `http://www.topografix.com/GPX/1/1`. Z każdego punktu odczytywane są atrybuty `lat` i `lon` oraz element `ele`. Współrzędne geograficzne i wysokość są konwertowane do typu zmiennoprzecinkowego.

Element `time` występuje w wygenerowanych plikach GPX, jednak w aktualnym zakresie programu nie jest wykorzystywany przez funkcje analityczne. Z tego powodu czas pomiaru nie jest przechowywany w strukturach zwracanych przez parser.

Do odczytu dokumentu XML wykorzystano moduł `xml.etree.ElementTree` z biblioteki standardowej Pythona [8]. W listingach pokazano wyłącznie fragmenty istotne dla logiki parsera, dlatego pominięto część importów bibliotek oraz elementy techniczne niewpływające bezpośrednio na sposób odczytu danych GPX.

Listing 3.2: Parser GPX w wariancie listowym

```
1 def parser_py(file):
2     ns = {"gpx": "http://www.topografix.com/GPX/1/1"}
3     tree = ET.parse(file)
4     root = tree.getroot()
5     trackpoints = root.findall("./gpx:trkpt", ns)
6     latitudes = []
7     longitudes = []
8     elevations = []
9     for trkpt in trackpoints:
10         lat_str = trkpt.get("lat")
11         lon_str = trkpt.get("lon")
12         ele_elem = trkpt.find("gpx:ele", ns)
13         if (
14             lat_str is None
15             or lon_str is None
16             or ele_elem is None
17             or not ele_elem.text
18         ):
19             continue
20         latitudes.append(float(lat_str))
21         longitudes.append(float(lon_str))
22         elevations.append(float(ele_elem.text))
23
24     return latitudes, longitudes, elevations
```

Listing 3.2 przedstawia parser GPX w wariancie opartym na listach Pythona. Początkowa część funkcji odpowiada za zdefiniowanie przestrzeni nazw GPX oraz wyszukanie w dokumencie wszystkich elementów `trkpt` (listing 3.2, linie 2–5). Uwzględnienie przestrzeni nazw jest konieczne, ponieważ punkty śladu w pliku GPX należą do domyślnej przestrzeni nazw dokumentu XML.

Zasadnicza część funkcji polega na przejściu po znalezionych punktach śladu. W tej pętli odczytywane są atrybuty `lat` i `lon` oraz element `ele`. Warunek sprawdzający kompletność tych wartości decyduje o pominięciu punktu niespełniającego założeń implementacji (listing 3.2, linie 13–19). Nie oznacza to, że brak wysokości w standardzie GPX jest równoznaczny z brakiem punktu geometrycznego; jest to decyzja wynikająca z zakresu analiz wykonywanych w programie.

Poprawnie odczytane wartości są konwertowane do typu zmiennoprzecinkowego i dopisywane do trzech osobnych list: szerokości geograficznych, długości geograficznych oraz wysokości. Taki układ danych jest zgodny z dalszymi funkcjami analitycznymi, które operują na wybranych składowych śladu.

Listing 3.3: Parser GPX w wariancie NumPy

```
1 def parser_np(file):
2     ns = {"gpx": "http://www.topografix.com/GPX/1/1"}
3     tree = ET.parse(file)
4     root = tree.getroot()
5     trackpoints = root.findall("./gpx:trkpt", ns)
6     n = len(trackpoints)
7     latitudes = np.empty(n, dtype=np.float64)
8     longitudes = np.empty(n, dtype=np.float64)
9     elevations = np.empty(n, dtype=np.float64)
10    i = 0
11    for trkpt in trackpoints:
12        lat_str = trkpt.get("lat")
13        lon_str = trkpt.get("lon")
14        ele_elem = trkpt.find("gpx:ele", ns)
15        if (
16            lat_str is None
17            or lon_str is None
18            or ele_elem is None
19            or not ele_elem.text
20        ):
21            continue
22        latitudes[i] = float(lat_str)
23        longitudes[i] = float(lon_str)
24        elevations[i] = float(ele_elem.text)
25        i += 1
26
27    return latitudes[:i], longitudes[:i], elevations[:i]
```

Listing 3.3 przedstawia parser przygotowany dla wariantu NumPy. Zakres odczytywanych danych pozostaje taki sam jak w wariancie listowym: funkcja wyszukuje punkty `trkpt`, odczytuje wartości `lat`, `lon` i `ele`, a punkty niespełniające tych założeń są pomijane.

Różnica dotyczy przede wszystkim sposobu organizacji pamięci dla wyników. Po

ustaleniu liczby znalezionych punktów tworzone są tablice typu `float64` (listing 3.3, linie 6–9). Wartości nie są więc dopisywane do list metodą `append`, lecz wpisywane do wcześniej zaalokowanych tablic.

Zmienna `i` pełni rolę licznika punktów faktycznie zapisanych w tablicach. Ma to znaczenie, ponieważ część punktów może zostać pominięta przez warunek sprawdzający kompletność danych. Z tego powodu funkcja zwraca wycinki tablic do indeksu `i`, a nie całe początkowo zaalokowane tablice (listing 3.3, linia 27).

Parser odczytuje punkty `trkpt` jako jedną uporządkowaną sekwencję. W benchmarkach wykorzystano pliki syntetyczne zawierające jeden segment `trkseg`, dlatego nie występuje problem łączenia punktów pochodzących z wielu segmentów śladu.

Parser nie odczytuje gotowych metryk zapisanych ewentualnie w rozszerzeniach pliku GPX. Dystans, suma podejść, suma zejść oraz miary porównania tras są wyznaczone ponownie na podstawie kolejnych punktów śladu.

3.3 Implementacja operacji analizy tras

Po odczytaniu pliku GPX dane wejściowe są przekazywane do funkcji analitycznych. W projekcie zaimplementowano cztery grupy operacji: obliczanie dystansu, wyznaczanie sumy podejść i zejść, porównywanie trasy zawodnika z trasą referencyjną oraz obliczanie dyskretnej odległości Fréchet’a.

Każda z tych operacji została przygotowana w trzech wariantach: listowym, NumPy oraz Numba. Warianty te zwracają ten sam typ wyniku dla danej operacji, ale różnią się organizacją danych i sposobem wykonania obliczeń.

3.3.1 Obliczanie dystansu i przewyższeń

Pierwszą grupę operacji stanowią metody wykonywane na pojedynczej trasie. Należą do nich obliczanie całkowitego dystansu oraz wyznaczanie sumy podejść i zejść. Obie operacje opierają się na relacji między sąsiednimi punktami śladu. Dystans jest obliczany jako suma odległości między kolejnymi parami punktów, natomiast przewyższenia wynikają z różnic wysokości między następującymi po sobie próbkami.

We wszystkich wariantach obliczania dystansu wykorzystano wzór Haversine’a jako podstawę wyznaczania odległości między kolejnymi punktami śladu. Dane wejściowe stanowią szerokości i długości geograficzne zapisane w stopniach, które przed wykonaniem obliczeń są przeliczane na radiany. W implementacji przyjęto promień Ziemi równy `6371000.0` metrów.

Wariant listowy zestawiono w listingu 3.4. Zawiera on funkcję obliczającą odległość między dwoma punktami, funkcję sumującą dystans dla całej trasy oraz funkcję wyznaczającą sumę podejść i zejść. Wariant ten wykorzystuje jawne pętle Pythona i stanowi

bezpośredni zapis algorytmów opisanych w części teoretycznej.

Listing 3.4: Obliczanie dystansu i przewyższeń w wariancie listowym

```
1 R = 6371000.0
2
3 def haversine_distance(lat1, lon1, lat2, lon2):
4     lat1_rad = math.radians(lat1)
5     lon1_rad = math.radians(lon1)
6     lat2_rad = math.radians(lat2)
7     lon2_rad = math.radians(lon2)
8     delta_lat = lat2_rad - lat1_rad
9     delta_lon = lon2_rad - lon1_rad
10    a = (
11        math.sin(delta_lat / 2) ** 2
12        + math.cos(lat1_rad)
13        * math.cos(lat2_rad)
14        * math.sin(delta_lon / 2) ** 2
15    )
16    a = min(1.0, max(0.0, a))
17    c = 2 * math.atan2(math.sqrt(a), math.sqrt(1 - a))
18    return R * c
19
20 def haversine_total_distance(latitudes, longitudes):
21     total_distance = 0.0
22     for i in range(1, len(latitudes)):
23         distance = haversine_distance(
24             latitudes[i - 1],
25             longitudes[i - 1],
26             latitudes[i],
27             longitudes[i],
28         )
29         total_distance += distance
30     return total_distance
31
32 def elevation_gain_loss(elevations):
33     elevation_gain = 0.0
34     elevation_loss = 0.0
35     for i in range(1, len(elevations)):
36         difference = elevations[i] - elevations[i - 1]
37         if difference > 0:
38             elevation_gain += difference
39         elif difference < 0:
40             elevation_loss += abs(difference)
41     return elevation_gain, elevation_loss
```

W funkcji `haversine_distance` najważniejszy fragment stanowi przeliczenie współ-

rzędnych na radiany oraz podstawienie ich do wzoru Haversine’a (listing 3.4, linie 3–18). Wartość pomocnicza `a` jest dodatkowo ograniczana do przedziału od 0 do 1 (listing 3.4, linia 16), co zabezpiecza dalsze obliczenia przed skutkami błędów zaokrągleń.

Całkowity dystans trasy jest wyznaczany w funkcji `haversine_total_distance`. Kluczowa jest tu pętla przechodząca po kolejnych parach sąsiednich punktów (listing 3.4, linie 20–30). Dla każdej pary wywoływana jest funkcja `haversine_distance`, a uzyskana odległość jest dodawana do sumy całkowitej.

Funkcja `elevation_gain_loss` wykorzystuje analogiczny schemat przejścia po sąsiednich próbkach, ale operuje na wysokościach zamiast na współrzędnych geograficznych. Fragment z liniami 35–40 odpowiada za obliczenie różnicy wysokości i rozdzielenie jej na sumę podejść oraz sumę zejść. Wartości zerowe nie wpływają na żaden z wyników.

Wariant NumPy realizuje te same obliczenia, lecz zmienia sposób przechodzenia po danych. Zamiast wywoływać funkcję odległości osobno dla każdej pary punktów, program przekazuje do funkcji Haversine’a przesunięte względem siebie fragmenty tablic. Dzięki temu odległości cząstkowe dla kolejnych par punktów mogą zostać obliczone jako operacja tablicowa.

Listing 3.5: Obliczanie dystansu i przewyższeń w wariancie NumPy

```
1 R = 6371000.0
2
3 def haversine_distance_np(lat1, lon1, lat2, lon2):
4     lat1_rad = np.radians(lat1)
5     lon1_rad = np.radians(lon1)
6     lat2_rad = np.radians(lat2)
7     lon2_rad = np.radians(lon2)
8     delta_lat = lat2_rad - lat1_rad
9     delta_lon = lon2_rad - lon1_rad
10    a = (
11        np.sin(delta_lat / 2.0) ** 2
12        + np.cos(lat1_rad)
13        * np.cos(lat2_rad)
14        * np.sin(delta_lon / 2.0) ** 2
15    )
16    a = np.clip(a, 0.0, 1.0)
17    c = 2.0 * np.arctan2(np.sqrt(a), np.sqrt(1.0 - a))
18    return R * c
19
20 def haversine_total_distance_np(latitudes, longitudes):
21     distances = haversine_distance_np(
22         latitudes[:-1],
23         longitudes[:-1],
24         latitudes[1:],
25         longitudes[1:],
```

```

26     )
27     return float(np.sum(distances))
28
29 def elevation_gain_loss_np(elevations):
30     differences = elevations[1:] - elevations[:-1]
31     elevation_gain = np.sum(differences[differences > 0.0])
32     elevation_loss = np.sum(np.abs(differences[differences < 0.0]))
33     return float(elevation_gain), float(elevation_loss)

```

Wariant tablicowy tych samych operacji pokazano w listingu 3.5. Wzór Haversine’a pozostaje taki sam jak w wariancie listowym, ale funkcje matematyczne pochodzą z biblioteki NumPy i mogą działać na całych tablicach wartości (listing 3.5, linie 3–18).

Najważniejsza różnica pojawia się w funkcji `haversine_total_distance_np`. Wycinki `latitudes[:-1]` i `latitudes[1:]` tworzą dwie przesunięte względem siebie sekwencje punktów (listing 3.5, linie 23–28). Analogiczne wycinki są wykonywane dla długości geograficznych. W efekcie funkcja Haversine’a otrzymuje wszystkie pary sąsiednich punktów jednocześnie, a wynikowa tablica odległości jest sumowana funkcją `np.sum`.

Podobnie zrealizowano obliczanie przewyższeń. Wyrażenie `elevations[1:] - elevations[:-1]` tworzy tablicę różnic wysokości między sąsiednimi punktami (listing 3.5, linia 34). Następnie maski logiczne wybierają wartości dodatnie i ujemne, które są sumowane jako suma podejść i suma zejść (listing 3.5, linie 35–36). Ten fragment jest przykładem operacji, która dobrze pasuje do jednowymiarowej reprezentacji tablicowej.

Trzeci wariant wykorzystuje bibliotekę Numba. W przeciwieństwie do wariantu NumPy nie próbuje on zapisać całego obliczenia jako operacji tablicowej. Struktura funkcji pozostaje zbliżona do wariantu listowego, ale najważniejsze pętle są umieszczone w funkcjach oznaczonych dekoratorem `@njit`. Dane wejściowe dla tego wariantu są przekazywane jako tablice NumPy.

Listing 3.6: Obliczanie dystansu i przewyższeń w wariancie Numba

```

1  R = 6371000.0
2
3  @njit
4  def haversine_distance_numba(lat1, lon1, lat2, lon2):
5      lat1_rad = math.radians(lat1)
6      lon1_rad = math.radians(lon1)
7      lat2_rad = math.radians(lat2)
8      lon2_rad = math.radians(lon2)
9      delta_lat = lat2_rad - lat1_rad
10     delta_lon = lon2_rad - lon1_rad
11     a = (
12         math.sin(delta_lat / 2.0) ** 2
13         + math.cos(lat1_rad)
14         * math.cos(lat2_rad)

```

```

15         * math.sin(delta_lon / 2.0) ** 2
16     )
17     if a < 0.0:
18         a = 0.0
19     elif a > 1.0:
20         a = 1.0
21     c = 2.0 * math.atan2(math.sqrt(a), math.sqrt(1.0 - a))
22     return R * c
23
24 @njit
25 def haversine_total_distance_numba(latitudes, longitudes):
26     total_distance = 0.0
27     for i in range(1, len(latitudes)):
28         distance = haversine_distance_numba(
29             latitudes[i - 1],
30             longitudes[i - 1],
31             latitudes[i],
32             longitudes[i],
33         )
34         total_distance += distance
35     return total_distance
36
37 @njit
38 def elevation_gain_loss_numba(elevations):
39     elevation_gain = 0.0
40     elevation_loss = 0.0
41     for i in range(1, len(elevations)):
42         difference = elevations[i] - elevations[i - 1]
43         if difference > 0.0:
44             elevation_gain += difference
45         elif difference < 0.0:
46             elevation_loss += abs(difference)
47     return elevation_gain, elevation_loss

```

Listing 3.6 przedstawia wariant wykorzystujący bibliotekę Numba. Dekorator `@njit` wskazuje funkcje, które mają zostać skompilowane (listing 3.6, linie 3, 24 i 37). Sama struktura obliczeń pozostaje zbliżona do wariantu listowego: całkowity dystans jest wyznaczany przez pętlę po kolejnych parach punktów, a przewyższenia przez pętlę po kolejnych wysokościach.

Najważniejsza różnica względem wariantu listowego dotyczy sposobu wykonania tych pętli. W funkcji `haversine_total_distance_numba` pętla po punktach znajduje się w części kompilowanej przez Numbę (listing 3.6, linie 27–35). Analogicznie funkcja `elevation_gain_loss_numba` zachowuje sekwencyjne porównywanie sąsiednich wysokości, ale wykonuje je w funkcji oznaczonej dekoratorem `@njit` (listing 3.6, linie 41–47).

Wariant Numba jest istotny porównawczo, ponieważ reprezentuje inne podejście niż NumPy. NumPy ogranicza udział jawnych pętli Pythona przez zapis tablicowy, natomiast Numba zachowuje strukturę pętli, ale zmienia mechanizm ich wykonania. Dzięki temu w dalszej analizie wyników można sprawdzić, czy dla operacji liniowych korzystniejsze okazały się operacje tablicowe, czy kompilacja pętli numerycznych.

Wszystkie trzy warianty zwracają ten sam typ wyniku: dla dystansu pojedynczą wartość liczbową wyrażoną w metrach, a dla przewyższeń parę wartości oznaczających sumę podejść i sumę zejść. Różnice między wariantami dotyczą więc sposobu wykonania obliczeń, a nie definicji metryki.

W implementacji przewyższeń nie zastosowano dodatkowego wygładzania profilu wysokościowego ani progów odrzucających bardzo małe zmiany wysokości. Funkcja realizuje więc bezpośrednią definicję sumowania dodatnich i ujemnych różnic wysokości między sąsiednimi punktami. Dzięki temu wszystkie warianty obliczeniowe wykonują tę samą operację, co ułatwia późniejsze porównanie czasu wykonania.

3.3.2 Porównywanie trasy zawodnika z trasą referencyjną

Kolejną grupę operacji stanowią metody porównujące dwie trasy: trasę referencyjną oraz ślad zawodnika. W odróżnieniu od obliczania dystansu i przewyższeń, które dotyczą pojedynczej sekwencji punktów, porównywanie tras wymaga analizy relacji między dwoma zbiorami danych. W przygotowanej implementacji przyjęto podejście polegające na obliczaniu odległości punktów śladu zawodnika od segmentów trasy referencyjnej.

Ponieważ współrzędne zapisane w plikach GPX są współrzędnymi geograficznymi wyrażonymi w stopniach, przed wykonaniem obliczeń geometrycznych są one przekształcane do układu metrycznego EPSG:2180. Dopiero w tym układzie możliwe jest bezpośrednie stosowanie odległości euklidesowej oraz interpretowanie wyników w metrach. Transformacja jest wykonywana z użyciem obiektu `Transformer` z biblioteki `pyproj` [9]. W kodzie zastosowano parametr `always_xy=True`, który wymusza kolejność argumentów zgodną z układem x, y . W przypadku współrzędnych geograficznych oznacza to przekazywanie najpierw długości geograficznej, a następnie szerokości geograficznej. Jest to istotne, ponieważ zamiana kolejności tych wartości prowadziłaby do błędnej transformacji współrzędnych i w konsekwencji do niepoprawnych odległości w układzie metrycznym.

Trasa referencyjna po transformacji jest traktowana jako linia łamana. Każda para sąsiednich punktów tworzy segment, względem którego można obliczać odległość punktu śladu zawodnika. Dla każdego punktu zawodnika wyznaczana jest minimalna odległość do wszystkich segmentów trasy referencyjnej. Następnie na podstawie uzyskanych odległości obliczana jest średnia odległość od trasy, największe odchylenie oraz odsetek punktów mieszczących się w przyjętych progach tolerancji.

Listing 3.7: Funkcje pomocnicze porównywania tras w wariancie listowym

```
1 def route_to_euclidean(latitudes, longitudes):
2     xs = []
3     ys = []
4     for lat, lon in zip(latitudes, longitudes):
5         x, y = TRANSFORMER_2180.transform(lon, lat)
6         xs.append(x)
7         ys.append(y)
8     return xs, ys
9
10 def build_segments(xs, ys):
11     segments = []
12     for i in range(1, len(xs)):
13         ax = xs[i - 1]
14         ay = ys[i - 1]
15         bx = xs[i]
16         by = ys[i]
17         abx = bx - ax
18         aby = by - ay
19         ab_len2 = abx * abx + aby * aby
20         segments.append((ax, ay, bx, by, abx, aby, ab_len2))
21     return segments
22
23 def distance_to_segments(px, py, segments):
24     min_dist2 = None
25     for ax, ay, bx, by, abx, aby, ab_len2 in segments:
26         apx = px - ax
27         apy = py - ay
28         if ab_len2 == 0.0:
29             dx = px - ax
30             dy = py - ay
31         else:
32             t = (apx * abx + apy * aby) / ab_len2
33             if t < 0.0:
34                 dx = px - ax
35                 dy = py - ay
36             elif t > 1.0:
37                 dx = px - bx
38                 dy = py - by
39             else:
40                 closest_x = ax + t * abx
41                 closest_y = ay + t * aby
42                 dx = px - closest_x
43                 dy = py - closest_y
44             dist2 = dx * dx + dy * dy
45             if min_dist2 is None or dist2 < min_dist2:
```

```
46         min_dist2 = dist2
47     return math.sqrt(min_dist2)
```

Funkcje pomocnicze wariantu listowego zestawiono w listingu 3.7. Pierwszym etapem jest transformacja współrzędnych geograficznych do układu EPSG:2180. Funkcja `route_to_euclidean` zapisuje wynik transformacji w dwóch osobnych listach `xs` oraz `ys` (listing 3.7, linie 1–8). Taki układ jest zgodny z ogólną reprezentacją składowych punktu przyjętą w projekcie.

Kolejnym krokiem jest budowa segmentów trasy referencyjnej. Funkcja `build_segments` przechodzi po sąsiednich punktach trasy i dla każdej pary zapisuje współrzędne początku i końca segmentu, składowe wektora oraz kwadrat długości segmentu (listing 3.7, linie 10–21). Kwadrat długości jest później wykorzystywany przy rzutowaniu punktu zawodnika na segment.

Najważniejszy fragment obliczeniowy znajduje się w funkcji `distance_to_segments`. Dla każdego segmentu wyznaczany jest parametr `t`, który określa położenie rzutu punktu względem segmentu (listing 3.7, linia 32). Następnie obsługiwane są trzy przypadki: rzut przed początkiem segmentu, rzut za końcem segmentu oraz rzut znajdujący się wewnątrz segmentu (listing 3.7, linie 33–43). Funkcja porównuje kwadraty odległości i dopiero na końcu zwraca pierwiastek z najmniejszej wartości.

Listing 3.8: Agregacja wyników porównania tras w wariacie listowym

```
1 def compare_routes(
2     reference_latitudes,
3     reference_longitudes,
4     runner_latitudes,
5     runner_longitudes,
6     thresholds_m=(5.0, 10.0, 20.0),
7 ):
8     reference_xs, reference_ys = route_to_euclidean(
9         reference_latitudes,
10        reference_longitudes,
11    )
12    runner_xs, runner_ys = route_to_euclidean(
13        runner_latitudes,
14        runner_longitudes,
15    )
16    segments = build_segments(reference_xs, reference_ys)
17    total_distance_from_route = 0.0
18    max_distance = 0.0
19    threshold_counts = {}
20    for threshold in thresholds_m:
21        threshold_counts[threshold] = 0
22    for i in range(len(runner_xs)):
```

```

23     distance = distance_to_segments(
24         runner_xs[i],
25         runner_ys[i],
26         segments,
27     )
28     total_distance_from_route += distance
29     if distance > max_distance:
30         max_distance = distance
31     for threshold in thresholds_m:
32         if distance <= threshold:
33             threshold_counts[threshold] += 1
34 points_count = len(runner_xs)
35 result = {
36     "mean_distance_m": total_distance_from_route / points_count,
37     "max_distance_m": max_distance,
38 }
39 for threshold in thresholds_m:
40     key = f"within_{int(threshold)}m_percent"
41     result[key] = threshold_counts[threshold] / points_count * 100.0
42 return result

```

Główną funkcję wariantu listowego pokazano w listingu 3.8. Na początku obie trasy są transformowane do układu metrycznego, a dla trasy referencyjnej budowana jest lista segmentów (listing 3.8, linie 8–16). Dopiero po tym etapie możliwe jest właściwe porównanie śladu zawodnika z trasą referencyjną.

Główna pętla funkcji przechodzi po punktach zawodnika i dla każdego z nich wyznacza minimalną odległość od segmentów trasy referencyjnej (listing 3.8, linie 22–33). W tej samej pętli aktualizowana jest suma odległości, największe odchylenie oraz liczniki punktów mieszczących się w zadanych progach tolerancji. Wynikiem funkcji jest słownik zawierający średnią odległość, maksymalną odległość oraz procent punktów mieszczących się w progach 5, 10 i 20 metrów.

Wariant NumPy wykorzystuje tę samą definicję odległości punktu od trasy referencyjnej, ale inaczej organizuje dane po transformacji współrzędnych. Współrzędne punktów oraz parametry segmentów są przechowywane w osobnych tablicach, co pozwala obliczać odległość jednego punktu zawodnika od wszystkich segmentów trasy referencyjnej jako operację tablicową.

Listing 3.9: Budowa segmentów i odległość punkt–segment w wariacie NumPy

```

1 def route_to_euclidean_np(latitudes, longitudes):
2     xs, ys = TRANSFORMER_2180.transform(longitudes, latitudes)
3     return xs, ys
4
5 def build_segments_np(xs, ys):

```

```

6     ax = xs[:-1]
7     ay = ys[:-1]
8     bx = xs[1:]
9     by = ys[1:]
10    abx = bx - ax
11    aby = by - ay
12    ab_len2 = abx * abx + aby * aby
13    return ax, ay, bx, by, abx, aby, ab_len2
14
15 def distance_to_segments_np(px, py, segments):
16     ax, ay, bx, by, abx, aby, ab_len2 = segments
17     apx = px - ax
18     apy = py - ay
19     t = np.zeros_like(ab_len2)
20     valid = ab_len2 != 0.0
21     t[valid] = (
22         apx[valid] * abx[valid]
23         + apy[valid] * aby[valid]
24     ) / ab_len2[valid]
25     t = np.clip(t, 0.0, 1.0)
26     closest_x = ax + t * abx
27     closest_y = ay + t * aby
28     dx = px - closest_x
29     dy = py - closest_y
30     dist2 = dx * dx + dy * dy
31     min_dist2 = np.min(dist2)
32     return float(np.sqrt(min_dist2))

```

Listing 3.9 pokazuje tablicową organizację obliczeń dla segmentów trasy referencyjnej. Funkcja `build_segments_np` nie tworzy listy krotek, lecz zestaw tablic opisujących początki segmentów, końce segmentów, wektory segmentów oraz kwadraty ich długości (listing 3.9, linie 5–13).

Kluczowy fragment częściowej wektoryzacji znajduje się w funkcji `distance_to_segments_np`. Dla jednego punktu zawodnika parametr `t` jest obliczany jednocześnie dla wszystkich segmentów referencyjnych (listing 3.9, linie 19–25). Ograniczenie wartości `t` do przedziału od 0 do 1 pozwala obsłużyć rzutowanie na segment bez osobnej pętli po segmentach. Następnie tablicowo wyznaczane są najbliższe punkty na segmentach oraz kwadraty odległości, z których wybierana jest wartość minimalna (listing 3.9, linie 26–32).

Listing 3.10: Porównywanie tras w wariancie NumPy

```

1 def compare_routes_np(
2     reference_latitudes,
3     reference_longitudes,
4     runner_latitudes,

```

```

5     runner_longitudes,
6     thresholds_m=(5.0, 10.0, 20.0),
7 ):
8     reference_xs, reference_ys = route_to_euclidean_np(
9         reference_latitudes,
10        reference_longitudes,
11    )
12    runner_xs, runner_ys = route_to_euclidean_np(
13        runner_latitudes,
14        runner_longitudes,
15    )
16    segments = build_segments_np(reference_xs, reference_ys)
17    distances = np.empty(len(runner_xs), dtype=np.float64)
18    for i in range(len(runner_xs)):
19        distances[i] = distance_to_segments_np(
20            runner_xs[i],
21            runner_ys[i],
22            segments,
23        )
24    result = {
25        "mean_distance_m": float(np.mean(distances)),
26        "max_distance_m": float(np.max(distances)),
27    }
28    for threshold in thresholds_m:
29        key = f"within_{int(threshold)}m_percent"
30        result[key] = float(
31            np.sum(distances <= threshold) / len(distances) * 100.0
32        )
33    return result

```

Listing 3.10 przedstawia funkcję główną wariantu NumPy. W odróżnieniu od operacji dystansu i przewyższeń metoda ta nie została w pełni zapisana jako jedna operacja tablicowa. Funkcja nadal przechodzi po punktach zawodnika w pętli (listing 3.10, linie 18–23), ponieważ dla każdego punktu trzeba wyznaczyć minimalną odległość od całej trasy referencyjnej.

Częściowa wektoryzacja dotyczy więc obliczenia odległości jednego punktu od wszystkich segmentów, a nie całego porównania obu tras jednocześnie. Po wypełnieniu tablicy `distances` średnia i maksymalna odległość są obliczane za pomocą funkcji `np.mean` i `np.max`, natomiast udział punktów mieszczących się w progach tolerancji jest wyznaczany z użyciem maski logicznej `distances <= threshold` (listing 3.10, linie 24–33).

Wariant Numba zachowuje strukturę obliczeń opartą na pętlach, ale przenosi główną część porównywania tras do funkcji kompilowanej. Transformacja współrzędnych z użyciem biblioteki `pyproj` pozostaje poza funkcją oznaczoną dekoratorem `@njit`, natomiast rdzeń numeryczny otrzymuje już tablice współrzędnych oraz tablice opisujące segmenty trasy

referencyjnej.

Listing 3.11: Rdzeń porównywania tras w wariancie Numba

```

1 @njit
2 def compare_routes_core_numba(
3     runner_xs,
4     runner_ys,
5     ax,
6     ay,
7     bx,
8     by,
9     abx,
10    aby,
11    ab_len2,
12    thresholds,
13 ):
14     total_distance_from_route = 0.0
15     max_distance = 0.0
16     threshold_counts = np.zeros(len(thresholds), dtype=np.int64)
17     for i in range(len(runner_xs)):
18         distance = distance_to_segments_numba(
19             runner_xs[i],
20             runner_ys[i],
21             ax,
22             ay,
23             bx,
24             by,
25             abx,
26             aby,
27             ab_len2,
28         )
29         total_distance_from_route += distance
30         if distance > max_distance:
31             max_distance = distance
32         for j in range(len(thresholds)):
33             if distance <= thresholds[j]:
34                 threshold_counts[j] += 1
35     mean_distance = total_distance_from_route / len(runner_xs)
36     return mean_distance, max_distance, threshold_counts

```

Listing 3.11 przedstawia skompilowany rdzeń wariantu Numba. Najważniejszy fragment znajduje się w pętli po punktach zawodnika (listing 3.11, linie 17–34). Dla każdego punktu wywoływana jest funkcja obliczająca minimalną odległość od segmentów referencyjnych, a następnie aktualizowane są suma odległości, maksymalne odchylenie i liczniki progów.

Warto zwrócić uwagę, że funkcja rdzeniowa nie buduje końcowego słownika wyników.

Zwraca wyłącznie wartości liczbowe oraz tablicę liczników (listing 3.11, linie 35–36). Dzięki temu część objęta kompilacją JIT pozostaje numeryczna i nie wymaga operowania na złożonych strukturach Pythona.

Listing 3.12: Funkcja zewnętrzna porównywania tras w wariancie Numba

```
1 def compare_routes_numba(  
2     reference_latitudes,  
3     reference_longitudes,  
4     runner_latitudes,  
5     runner_longitudes,  
6     thresholds_m=(5.0, 10.0, 20.0),  
7 ):  
8     reference_xs, reference_ys = route_to_euclidean_numba(  
9         reference_latitudes,  
10        reference_longitudes,  
11    )  
12    runner_xs, runner_ys = route_to_euclidean_numba(  
13        runner_latitudes,  
14        runner_longitudes,  
15    )  
16    ax, ay, bx, by, abx, aby, ab_len2 = build_segments_numba(  
17        reference_xs,  
18        reference_ys,  
19    )  
20    thresholds = np.asarray(thresholds_m, dtype=np.float64)  
21    mean_distance, max_distance, threshold_counts = compare_routes_core_numba(  
22        runner_xs,  
23        runner_ys,  
24        ax,  
25        ay,  
26        bx,  
27        by,  
28        abx,  
29        aby,  
30        ab_len2,  
31        thresholds,  
32    )  
33    result = {  
34        "mean_distance_m": float(mean_distance),  
35        "max_distance_m": float(max_distance),  
36    }  
37    for i in range(len(thresholds)):  
38        key = f"within_{int(thresholds[i])}m_percent"  
39        result[key] = float(threshold_counts[i] / len(runner_xs) * 100.0)  
40    return result
```

Listing 3.12 przedstawia funkcję zewnętrzną wariantu Numba. Jej zadaniem jest przygotowanie danych dla funkcji skompilowanej: transformacja obu tras do układu EPSG:2180, budowa segmentów trasy referencyjnej oraz konwersja progów tolerancji do tablicy NumPy (listing 3.12, linie 8–20).

Właściwe porównanie tras jest wykonywane przez funkcję `compare_routes_core_numba` (listing 3.12, linie 21–33). Dopiero po jej zakończeniu funkcja zewnętrzna tworzy słownik wynikowy w takim samym układzie jak wariant listowy i NumPy. Takie rozdzielanie pokazuje, że w wariacie Numba kompilowany jest przede wszystkim rdzeń numeryczny, a nie cały proces od danych wejściowych do końcowej struktury wyniku.

Wszystkie trzy warianty implementacyjne zwracają wynik w tej samej postaci. Słownik wynikowy zawiera średnią odległość punktów śladu zawodnika od trasy referencyjnej, maksymalne odchylenie oraz procent punktów mieszczących się w progach tolerancji 5, 10 i 20 metrów. Dzięki temu wyniki wariantów listowego, NumPy i Numba mogą być porównywane zarówno pod względem wartości obliczonych metryk, jak i czasu wykonania.

Z punktu widzenia złożoności obliczeniowej metoda wymaga porównania każdego punktu śladu zawodnika z segmentami trasy referencyjnej. Jeżeli trasa zawodnika zawiera m punktów, a trasa referencyjna s segmentów, liczba podstawowych relacji punkt–segment wynosi $m \cdot s$. Wariant NumPy należy traktować jako częściowo zwektoryzowany: operacje względem segmentów są wykonywane tablicowo dla jednego punktu zawodnika, lecz funkcja nadal iteruje po kolejnych punktach śladu. Wariant Numba zachowuje pętle, ale przenosi ich wykonanie do funkcji kompilowanej.

3.3.3 Obliczanie dyskretnej odległości Frécheta

Ostatnią zaimplementowaną metodą porównywania tras jest dyskretna odległość Frécheta. W przeciwieństwie do metody punkt–segment, która dla każdego punktu śladu zawodnika wyznacza minimalną odległość od trasy referencyjnej, dyskretna odległość Frécheta porównuje dwie sekwencje punktów jako całość. Metoda ta uwzględnia kolejność punktów w obu trasach, dlatego opisuje podobieństwo przebiegu dwóch linii łamanych w sposób zależny od uporządkowania punktów.

Podobnie jak w przypadku porównywania punktów z segmentami, przed wykonaniem obliczeń współrzędne geograficzne są przekształcane do układu EPSG:2180. Dzięki temu odległość między punktami może być obliczana jako odległość euklidesowa w metrach. Po transformacji obie trasy są traktowane jako sekwencje punktów płaskich. Następnie tworzone jest macierz programowania dynamicznego, w której komórka odpowiada porównaniu początkowych fragmentów obu sekwencji kończących się w danej parze punktów.

W implementacji zastosowano pełną macierz programowania dynamicznego o wymiarach n na m , gdzie n oznacza liczbę punktów trasy referencyjnej, a m liczbę punktów trasy zawodnika. Oznacza to, że metoda ma zarówno koszt czasowy, jak i pamięciowy zależny

od iloczynu liczby punktów obu tras.

Listing 3.13: Dyskretna odległość Fréchet’a w wariancie listowym

```

1 def frechet_distance(
2     reference_latitudes,
3     reference_longitudes,
4     runner_latitudes,
5     runner_longitudes,
6 ):
7     reference_xs, reference_ys = route_to_euclidean(
8         reference_latitudes,
9         reference_longitudes,
10    )
11    runner_xs, runner_ys = route_to_euclidean(
12        runner_latitudes,
13        runner_longitudes,
14    )
15    n = len(reference_xs)
16    m = len(runner_xs)
17    ca = [[0.0 for _ in range(m)] for _ in range(n)]
18    for i in range(n):
19        for j in range(m):
20            dx = reference_xs[i] - runner_xs[j]
21            dy = reference_ys[i] - runner_ys[j]
22            dist2 = dx * dx + dy * dy
23            if i == 0 and j == 0:
24                ca[i][j] = dist2
25            elif i == 0:
26                ca[i][j] = max(ca[i][j - 1], dist2)
27            elif j == 0:
28                ca[i][j] = max(ca[i - 1][j], dist2)
29            else:
30                ca[i][j] = max(
31                    min(
32                        ca[i - 1][j],
33                        ca[i - 1][j - 1],
34                        ca[i][j - 1],
35                    ),
36                    dist2,
37                )
38    return math.sqrt(ca[n - 1][m - 1])

```

Listing 3.13 przedstawia wariant listowy obliczania dyskretnej odległości Fréchet’a. Pierwszym etapem jest transformacja obu tras do układu metrycznego (listing 3.13, linie 7–14). Funkcja nie operuje więc bezpośrednio na szerokościach i długościach geograficznych,

lecz na dwóch listach współrzędnych **xs** oraz **ys** dla każdej z tras.

Kluczowa struktura danych tworzona jest w linii 17. Jest to dwuwymiarowa lista **ca**, pełniąca rolę macierzy programowania dynamicznego. Jej rozmiar zależy od liczby punktów obu porównywanych tras. Zasadnicze obliczenia wykonywane są w dwóch zagnieżdżonych pętlach (listing 3.13, linie 18–37), które przechodzą po wszystkich parach punktów trasy referencyjnej i trasy zawodnika.

W liniach 20–22 obliczany jest kwadrat odległości euklidesowej między aktualną parą punktów. W macierzy zapisywana jest wartość **dist2**, a nie pierwiastek z tej wartości. Jest to możliwe, ponieważ porównywanie odległości w rekurencji odbywa się przez operacje minimum i maksimum, a pierwiastek kwadratowy jest funkcją monotoniczną dla wartości nieujemnych. Końcowy pierwiastek jest obliczany dopiero przy zwracaniu wyniku funkcji.

Listing 3.14: Dyskretna odległość Fréchet’a w wariancie NumPy

```

1 def frechet_distance_np(
2     reference_latitudes,
3     reference_longitudes,
4     runner_latitudes,
5     runner_longitudes,
6 ):
7     reference_xs, reference_ys = route_to_euclidean_np(
8         reference_latitudes,
9         reference_longitudes,
10    )
11    runner_xs, runner_ys = route_to_euclidean_np(
12        runner_latitudes,
13        runner_longitudes,
14    )
15    n = len(reference_xs)
16    m = len(runner_xs)
17    ca = np.zeros((n, m), dtype=np.float64)
18    for i in range(n):
19        for j in range(m):
20            dx = reference_xs[i] - runner_xs[j]
21            dy = reference_ys[i] - runner_ys[j]
22            dist2 = dx * dx + dy * dy
23            if i == 0 and j == 0:
24                ca[i, j] = dist2
25            elif i == 0:
26                ca[i, j] = max(ca[i, j - 1], dist2)
27            elif j == 0:
28                ca[i, j] = max(ca[i - 1, j], dist2)
29            else:
30                ca[i, j] = max(
31                    min(

```

```

32         ca[i - 1, j],
33         ca[i - 1, j - 1],
34         ca[i, j - 1],
35     ),
36     dist2,
37 )
38 return float(np.sqrt(ca[n - 1, m - 1]))

```

Wariant NumPy tej samej metody zaprezentowano w listingu 3.14. Zakres obliczeń pozostaje taki sam jak w wariancie listowym: obie trasy są transformowane do układu EPSG:2180, a następnie porównywane za pomocą macierzy programowania dynamicznego. Różnica dotyczy reprezentacji tej macierzy. W wariancie NumPy macierz `ca` jest tworzona jako tablica `ndarray` typu `float64` (listing 3.14, linia 17), a nie jako lista list.

Warto podkreślić, że zastosowanie NumPy w tym miejscu nie oznacza pełnej wektoryzacji algorytmu. Macierz nadal jest wypełniana iteracyjnie w dwóch zagnieżdżonych pętlach (listing 3.14, linie 18–37). Wynika to z zależności rekurencyjnej: wartość bieżącej komórki zależy od wcześniej obliczonych komórek po lewej, powyżej oraz po przekątnej. NumPy pełni tutaj przede wszystkim rolę jednorodnej reprezentacji macierzy numerycznej.

Listing 3.15: Rdzeń obliczania dyskretnej odległości Fréchet’a w wariancie Numba

```

1 @njit
2 def frechet_distance_xy_numba(reference_xs, reference_ys, runner_xs, runner_ys):
3     n = len(reference_xs)
4     m = len(runner_xs)
5     ca = np.zeros((n, m), dtype=np.float64)
6     for i in range(n):
7         for j in range(m):
8             dx = reference_xs[i] - runner_xs[j]
9             dy = reference_ys[i] - runner_ys[j]
10            dist2 = dx * dx + dy * dy
11            if i == 0 and j == 0:
12                ca[i, j] = dist2
13            elif i == 0:
14                ca[i, j] = max(ca[i, j - 1], dist2)
15            elif j == 0:
16                ca[i, j] = max(ca[i - 1, j], dist2)
17            else:
18                ca[i, j] = max(
19                    min(
20                        ca[i - 1, j],
21                        ca[i - 1, j - 1],
22                        ca[i, j - 1],
23                    ),
24                    dist2,

```

```

25         )
26     return math.sqrt(ca[n - 1, m - 1])

```

Listing 3.15 przedstawia rdzeń obliczeniowy wariantu Numba. Funkcja otrzymuje już współrzędne w układzie metrycznym, dlatego nie obejmuje etapu transformacji z wykorzystaniem biblioteki `pyproj`. Kluczowe znaczenie ma dekorator `@jit` (listing 3.15, linia 1), który wskazuje, że funkcja ma zostać skompilowana przez Numbę.

Sama struktura algorytmu pozostaje taka sama jak w wariacie NumPy: tworzona jest macierz `ca`, a następnie dwie zagnieżdżone pętle przechodzą po wszystkich parach punktów (listing 3.15, linie 5–25). Różnica polega na sposobie wykonania tych pętli. Wariant Numba nie usuwa zależności rekurencyjnej, lecz przenosi jej wykonanie do funkcji kompilowanej.

Listing 3.16: Funkcja zewnętrzna obliczania odległości Frécheta w wariacie Numba

```

1  def frechet_distance_numba(
2      reference_latitudes,
3      reference_longitudes,
4      runner_latitudes,
5      runner_longitudes,
6  ):
7      reference_xs, reference_ys = route_to_euclidean_numba(
8          reference_latitudes,
9          reference_longitudes,
10     )
11     runner_xs, runner_ys = route_to_euclidean_numba(
12         runner_latitudes,
13         runner_longitudes,
14     )
15     return float(
16         frechet_distance_xy_numba(
17             reference_xs,
18             reference_ys,
19             runner_xs,
20             runner_ys,
21         )
22     )

```

Listing 3.16 przedstawia funkcję zewnętrzną wariantu Numba. Jej zadaniem jest wykonanie transformacji obu tras do układu EPSG:2180 (listing 3.16, linie 7–14), a następnie przekazanie przygotowanych tablic współrzędnych do funkcji rdzeniowej (listing 3.16, linie 15–22). Takie rozdzielenie jest istotne, ponieważ kompilacją JIT objęty jest numeryczny rdzeń algorytmu, a nie cały proces przygotowania danych.

Wszystkie trzy warianty implementują tę samą zależność programowania dynamicznego i zwracają pojedynczą wartość odległości w metrach. Różnice między nimi dotyczą

sposobu reprezentacji danych oraz wykonania pętli. Wariant listowy używa osobnych list współrzędnych oraz listy list jako macierzy DP. Wariant NumPy wykorzystuje tablicę `ndarray`, ale nadal wypełnia macierz iteracyjnie. Wariant Numba zachowuje strukturę iteracyjną, lecz wykonuje główną pętlę w funkcji kompilowanej.

Ze względu na konieczność utworzenia macierzy o wymiarach `n` na `m`, metoda ta jest znacznie bardziej kosztowna pamięciowo niż obliczanie dystansu, przewyższeń lub lokalnych odległości punkt-segment. Dla dwóch tras zawierających po 10 000 punktów macierz programowania dynamicznego zawiera 100 000 000 komórek. W przypadku tablicy typu `float64` oznacza to około 800 MB danych wyłącznie na samą macierz, bez uwzględnienia pozostałych struktur pomocniczych. Wariant listowy może mieć jeszcze większy narzut pamięciowy, ponieważ lista list w Pythonie przechowuje referencje do obiektów i nie jest jednorodnym buforem wartości typu `float64`.

3.4 Implementacja benchmarków

Ostatnim elementem implementacji są skrypty benchmarkowe służące do pomiaru czasu wykonania poszczególnych operacji analitycznych. W projekcie przygotowano osobne skrypty dla każdej grupy metod: obliczania dystansu, obliczania przewyższeń, porównywania trasy zawodnika z trasą referencyjną oraz obliczania dyskretnej odległości Frécheta. Taki podział pozwala analizować funkcje o różnej strukturze obliczeniowej niezależnie od siebie.

Benchmarki nie mierzą czasu parsowania pliku GPX. Każdy skrypt najpierw odczytuje dane wejściowe do struktur odpowiednich dla danego wariantu implementacyjnego, a dopiero następnie mierzy czas wykonania właściwej funkcji analitycznej. Oznacza to, że wyniki benchmarków odnoszą się do kosztu obliczeń wykonywanych na przygotowanych danych, a nie do całkowitego czasu przetwarzania pliku od momentu odczytu dokumentu XML. Takie rozdzielenie jest istotne, ponieważ celem pracy jest porównanie wariantów obliczeniowych, a nie porównanie wydajności parsera XML.

Do pomiaru czasu wykorzystano funkcję `time.perf_counter`. W każdym skrypcie pomiarowym zastosowano funkcję pomocniczą `measure`, która wykonuje przebiegi rozgrzewkowe, serię pomiarów właściwych oraz zwraca średni czas wykonania i listę czasów pojedynczych przebiegów.

Listing 3.17: Funkcja pomocnicza do pomiaru czasu wykonania

```
1 def measure(function, *args):
2     for _ in range(WARMUP_RUNS):
3         function(*args)
4     times = []
5     last_result = None
6     for _ in range(MEASURED_RUNS):
```

```
7     start = time.perf_counter()
8     last_result = function(*args)
9     elapsed_time = time.perf_counter() - start
10    times.append(elapsed_time)
11    return {
12        "czas": sum(times) / len(times),
13        "czasy": times,
14        "wynik": last_result,
15    }
```

Listing 3.17 przedstawia wspólny schemat pomiaru czasu. Przebiegi rozgrzewkowe wykonywane są przed właściwą serią pomiarów (listing 3.17, linie 2–3). Ich celem jest ograniczenie wpływu jednorazowych kosztów przygotowania funkcji lub danych na wynik końcowy. Właściwe pomiary wykonywane są w pętli z liniami 6–10, gdzie dla każdego przebiegu zapisywany jest czas rozpoczęcia, wywoływana jest badana funkcja, a następnie obliczany jest czas trwania wywołania.

Funkcja zwraca średni czas wykonania, listę czasów cząstkowych oraz wynik ostatniego wywołania (listing 3.17, linie 11–15). Pole **wynik** może być wykorzystane pomocniczo do sprawdzenia, czy wariant listowy, NumPy i Numba zwracają zgodne wartości dla tych samych danych wejściowych. Nie musi ono jednak być zapisywane w pliku JSON z wynikami benchmarku.

Dla operacji wykonywanych na pojedynczej trasie, takich jak obliczanie dystansu oraz sumowanie przewyższeń, zastosowano większe zbiory danych. W tych przypadkach koszt obliczeniowy rośnie liniowo wraz z liczbą punktów, dlatego benchmarki obejmują pliki od `trasa_10000.gpx` do `trasa_1000000.gpx`. Dla tych operacji wykonano trzy przebiegi rozgrzewkowe oraz dziesięć przebiegów pomiarowych.

Listing 3.18: Zakres danych i liczba pomiarów dla operacji liniowych

```
1 FILES = [
2     "data/trasa_10000.gpx",
3     "data/trasa_25000.gpx",
4     "data/trasa_50000.gpx",
5     "data/trasa_100000.gpx",
6     "data/trasa_250000.gpx",
7     "data/trasa_500000.gpx",
8     "data/trasa_1000000.gpx",
9 ]
10 WARMUP_RUNS = 3
11 MEASURED_RUNS = 10
```

Listing 3.18 pokazuje zakres danych wykorzystywany dla operacji liniowych. Pliki wejściowe zapisane w liniach 2–8 różnią się liczbą punktów śladu, natomiast linie 10–11

określają liczbę przebiegów rozgrzewkowych i pomiarowych. Taki zakres jest uzasadniony dla dystansu i przewyższeń, ponieważ obie operacje wymagają jedynie przejścia po sekwencji punktów.

Inaczej potraktowano metody porównujące dwie trasy. Zarówno porównywanie punktów śladu zawodnika z segmentami trasy referencyjnej, jak i obliczanie dyskretnej odległości Frécheta są znacznie bardziej kosztowne niż operacje liniowe. W pierwszej metodzie liczba sprawdzeń rośnie wraz z iloczynem liczby punktów zawodnika i liczby segmentów trasy referencyjnej. W drugiej metodzie tworzona jest macierz programowania dynamicznego, której liczba komórek jest iloczynem liczby punktów obu tras.

Dla benchmarku porównywania tras oraz dla benchmarku dyskretnej odległości Frécheta przyjęto te same pary plików wejściowych. W każdym przypadku plik `trasa_N.gpx` jest porównywany z plikiem `trasa_N_zawodnik.gpx`, gdzie `N` przyjmuje wartości od 1000 do 10000 co 1000 punktów. Takie ustawienie ułatwia późniejsze porównanie trendów czasu wykonania dla obu metod działających na dwóch trasach.

Listing 3.19: Zakres danych dla benchmarków metod porównujących dwie trasy

```
1 ROUTE_PAIRS = [  
2     ("data/trasa_1000.gpx", "data/trasa_1000_zawodnik.gpx"),  
3     ("data/trasa_2000.gpx", "data/trasa_2000_zawodnik.gpx"),  
4     ("data/trasa_3000.gpx", "data/trasa_3000_zawodnik.gpx"),  
5     ("data/trasa_4000.gpx", "data/trasa_4000_zawodnik.gpx"),  
6     ("data/trasa_5000.gpx", "data/trasa_5000_zawodnik.gpx"),  
7     ("data/trasa_6000.gpx", "data/trasa_6000_zawodnik.gpx"),  
8     ("data/trasa_7000.gpx", "data/trasa_7000_zawodnik.gpx"),  
9     ("data/trasa_8000.gpx", "data/trasa_8000_zawodnik.gpx"),  
10    ("data/trasa_9000.gpx", "data/trasa_9000_zawodnik.gpx"),  
11    ("data/trasa_10000.gpx", "data/trasa_10000_zawodnik.gpx"),  
12 ]  
13 WARMUP_RUNS = 2  
14 MEASURED_RUNS = 5
```

Listing 3.19 przedstawia zakres danych stosowany dla metod porównujących dwie trasy. W liniach 2–11 zapisano pary plików wejściowych, przy czym każda para zawiera trasę referencyjną oraz odpowiadającą jej trasę zawodnika o tej samej liczbie punktów. Linie 13–14 określają liczbę przebiegów rozgrzewkowych i pomiarowych. Takie same ustawienia zastosowano w benchmarku porównywania punktów z segmentami oraz w benchmarku dyskretnej odległości Frécheta.

Należy jednak zaznaczyć, że mimo tego samego zakresu danych obie metody mają inną strukturę obliczeniową. Porównywanie punktów z segmentami wymaga sprawdzenia relacji punkt–segment, natomiast dyskretna odległość Frécheta wymaga utworzenia macierzy programowania dynamicznego. Z tego powodu wyniki czasowe tych metod należy

interpretować oddzielnie.

W przypadku funkcji kompilowanych z użyciem Numby istotne jest oddzielenie kosztu kompilacji JIT od właściwego czasu wykonania funkcji. Oprócz przebiegów rozgrzewkowych na danych z danego benchmarku wykonywane jest również krótkie wywołanie wariantu Numba na fragmencie danych. Celem tego wywołania jest wymuszenie kompilacji funkcji oznaczonych dekoratorem `@njit`, a nie uzyskanie wyniku pomiarowego.

Listing 3.20: Przykład krótkiego wywołania wymuszającego kompilację Numby

```
1 frechet_distance_numba(  
2     ref_lat_np[:10],  
3     ref_lon_np[:10],  
4     run_lat_np[:10],  
5     run_lon_np[:10],  
6 )
```

Listing 3.20 pokazuje przykład wywołania kompilacyjnego dla wariantu Numba. Do funkcji przekazywany jest tylko krótki fragment danych wejściowych, ograniczony do pierwszych dziesięciu punktów każdej trasy (listing 3.20, linie 2–5). Wynik tego wywołania nie jest zapisywany jako wynik benchmarku. Właściwy pomiar czasu rozpoczyna się dopiero po zakończeniu takiego wywołania przygotowawczego.

Każdy skrypt benchmarkowy zapisuje wyniki do osobnego pliku JSON w katalogu **results**. Oddzielne pliki wynikowe utworzono dla dystansu, przewyższeń, porównywania tras oraz dyskretnej odległości Frécheta. Dzięki temu wyniki metod o różnych kosztach obliczeniowych nie są mieszane w jednym zbiorze danych.

Przykładowy rekord wynikowy zawiera nazwę pliku wejściowego, liczbę punktów, nazwę operacji, liczbę przebiegów rozgrzewkowych, liczbę przebiegów pomiarowych, średni czas wykonania dla każdego wariantu oraz listy czasów pojedynczych pomiarów. W przypadku metod porównujących dwie trasy zapisywane są dodatkowo nazwy obu plików wejściowych oraz liczba relacji punkt–segment albo liczba komórek macierzy programowania dynamicznego.

Listing 3.21: Przykładowa struktura rekordu wynikowego benchmarku

```
1 record = {  
2     "plik": file,  
3     "liczba_punktow": len(lat),  
4     "operacja": "dystans",  
5     "liczba_warmupow": WARMUP_RUNS,  
6     "liczba_pomiarow": MEASURED_RUNS,  
7     "sredni_czas_lista": list_result["czas"],  
8     "sredni_czas_numpy": numpy_result["czas"],  
9     "sredni_czas_numba": numba_result["czas"],  
10    "czasy_lista": list_result["czasy"],
```

```
11 |     "czasy_numpy": numpy_result["czasy"],  
12 |     "czasy_numba": numba_result["czasy"],  
13 | }
```

Listing 3.21 przedstawia przykładową strukturę rekordu wynikowego dla operacji wykonywanej na pojedynczej trasie. Linie 2–6 opisują dane identyfikujące pomiar: nazwę pliku, liczbę punktów, nazwę operacji oraz liczbę przebiegów rozgrzewkowych i pomiarowych. Linie 7–12 zawierają średnie czasy wykonania oraz listy czasów cząstkowych dla wariantu listowego, NumPy i Numba. Zapis list czasów pozwala w rozdziale wynikowym analizować nie tylko wartość średnią, lecz także stabilność pomiarów.

Opisane skrypty benchmarkowe kończą część implementacyjną pracy. W rozdziale tym przedstawiono pełny przepływ danych: od wygenerowania syntetycznych plików GPX, przez parsowanie punktów śladu, po wykonanie funkcji analitycznych w trzech wariantach implementacyjnych. Same wartości czasów wykonania nie są omawiane w tym miejscu, ponieważ stanowią przedmiot osobnego rozdziału poświęconego metodyce eksperymentów i analizie wyników.

Rozdział 4

Metodyka eksperymentów i analiza wyników

W niniejszym rozdziale przedstawiono sposób przeprowadzenia eksperymentów wydajnościowych oraz wyniki benchmarków uzyskane dla zaimplementowanych wariantów obliczeniowych. Analizie poddano cztery grupy operacji: obliczanie dystansu, obliczanie przewyższeń, porównywanie trasy zawodnika z trasą referencyjną oraz obliczanie dyskretnej odległości Frécheta.

Celem eksperymentów było porównanie czasu wykonania tych samych operacji zrealizowanych w trzech wariantach: bazowym opartym na listach Pythona, wariancie wykorzystującym bibliotekę NumPy oraz wariancie wykorzystującym bibliotekę Numba i kompilację JIT. W rozdziale tym nie analizuje się już implementacji funkcji, lecz ich zachowanie pomiarowe dla rosnącego rozmiaru danych wejściowych.

Wyniki prezentowane są w postaci tabel oraz wykresów. Tabele zawierają średnie czasy wykonania uzyskane w kolejnych benchmarkach, natomiast wykresy mają ułatwiać ocenę trendu wzrostu czasu wykonania oraz różnic między badanymi wariantami.

4.1 Środowisko testowe

Wiarygodna interpretacja benchmarków wymaga określenia środowiska, w którym zostały wykonane pomiary. Czas działania funkcji może zależeć między innymi od procesora, pamięci operacyjnej, systemu operacyjnego, wersji interpretera Pythona oraz środowiska, w którym uruchamiane są skrypty. Najważniejsze parametry sprzętowe i programowe zestawiono w tabeli 4.1.

Tabela 4.1: Podstawowe parametry środowiska testowego

Parametr	Wartość
System operacyjny hosta	Microsoft Windows 10 Pro
Typ komputera	Komputer stacjonarny x64
Procesor	Intel Core i7-5820K CPU @ 3.30 GHz
Liczba rdzeni procesora	6 rdzeni
Pamięć RAM	32 GB
Płyta główna	MSI X99S SLI PLUS (MS-7885)
Środowisko uruchomieniowe	Windows Subsystem for Linux 2
Wersja WSL	2.7.3.0
Domyślna dystrybucja WSL	Ubuntu
Dystrybucja Linux	Ubuntu 24.04.4 LTS (Noble Numbat)
Wersja jądra Linux w WSL	6.6.114.1-microsoft-standard-WSL2
Wersja interpretera Pythona	Python 3.12.3

Skrypty benchmarkowe były uruchamiane w środowisku WSL 2, w dystrybucji Ubuntu. Parametry sprzętowe odnoszą się do komputera hosta, natomiast pomiary wykonywano w środowisku linuksowym udostępnionym przez WSL. Takie rozróżnienie jest istotne, ponieważ system hosta i środowisko wykonania skryptów nie są w tym przypadku tym samym systemem operacyjnym.

4.2 Metodyka pomiarów

Pomiary wykonano w celu porównania czasu działania czterech grup funkcji analitycznych: obliczania dystansu, obliczania przewyższeń, porównywania trasy zawodnika z trasą referencyjną oraz obliczania dyskretnej odległości Fréchet’a. Każda z tych operacji została uruchomiona w trzech wariantach implementacyjnych: listowym, NumPy oraz Numba.

Dane wejściowe zostały przygotowane za pomocą skryptu `generate_gpx_data.py`. Skrypt tworzy syntetyczne pliki GPX o ustalonej liczbie punktów. Dla każdej wartości N powstają dwa pliki: `trasa_N.gpx` oraz `trasa_N_zawodnik.gpx`. Pierwszy z nich pełni rolę trasy referencyjnej, natomiast drugi zawiera niewielkie odchylenia współrzędnych i jest używany w metodach porównujących dwie trasy.

Wygenerowane pliki zachowują strukturę GPX 1.1 i zawierają jeden segment `trkseg`. Współrzędne punktów są zapisane jako szerokość i długość geograficzna w stopniach dziesiętnych, a nie jako współrzędne w układzie metrycznym. Dzięki temu dane wejściowe odpowiadają strukturze plików GPX, natomiast ewentualna transformacja do układu metrycznego jest wykonywana dopiero w funkcjach wymagających obliczeń geometrycznych.

Benchmarki obejmowały wyłącznie czas wykonania funkcji analitycznych na danych wcześniej odczytanych z plików GPX. Etap parsowania dokumentu XML nie był wli-

czany do czasu właściwych obliczeń. Takie rozdzielenie pozwala porównywać warianty implementacyjne funkcji, a nie wydajność samego parsera.

Tabela 4.2: Zakres danych i liczba powtórzeń w benchmarkach

Operacja	Zakres danych	Warm-up	Pomiary
Obliczanie dystansu	10 000–1 000 000 punktów	3	10
Obliczanie przewyższeń	10 000–1 000 000 punktów	3	10
Porównywanie tras	pary tras od 1 000 do 10 000 punktów, co 1 000	2	5
Dyskretna odległość Fréchet	pary tras od 1 000 do 10 000 punktów, co 1 000	2	5

Do pomiaru czasu wykorzystano funkcję `time.perf_counter`. Dla wariantu Numba przed właściwym pomiarem wykonywano krótkie wywołanie przygotowawcze, aby oddzielić koszt kompilacji JIT od czasu działania funkcji już skompilowanej. Wyniki zapisywano do plików JSON w katalogu `results`, osobno dla każdej grupy operacji.

4.3 Wyniki benchmarków

Wyniki benchmarków przedstawiono osobno dla każdej grupy operacji. Taki układ pozwala porównać warianty implementacyjne w ramach funkcji o podobnej strukturze obliczeniowej. Operacje wykonywane na pojedynczej trasie, takie jak dystans i przewyższenia, zestawiono oddzielnie od metod porównujących dwie trasy, ponieważ różnią się one kosztem obliczeniowym oraz sposobem organizacji pętli.

W każdej podsekcji zaprezentowano tabelę ze średnimi czasami wykonania oraz wykres pokazujący zmianę czasu wraz ze wzrostem liczby punktów. Interpretacja wyników dotyczy badanej implementacji i przyjętego środowiska pomiarowego.

4.3.1 Obliczanie dystansu

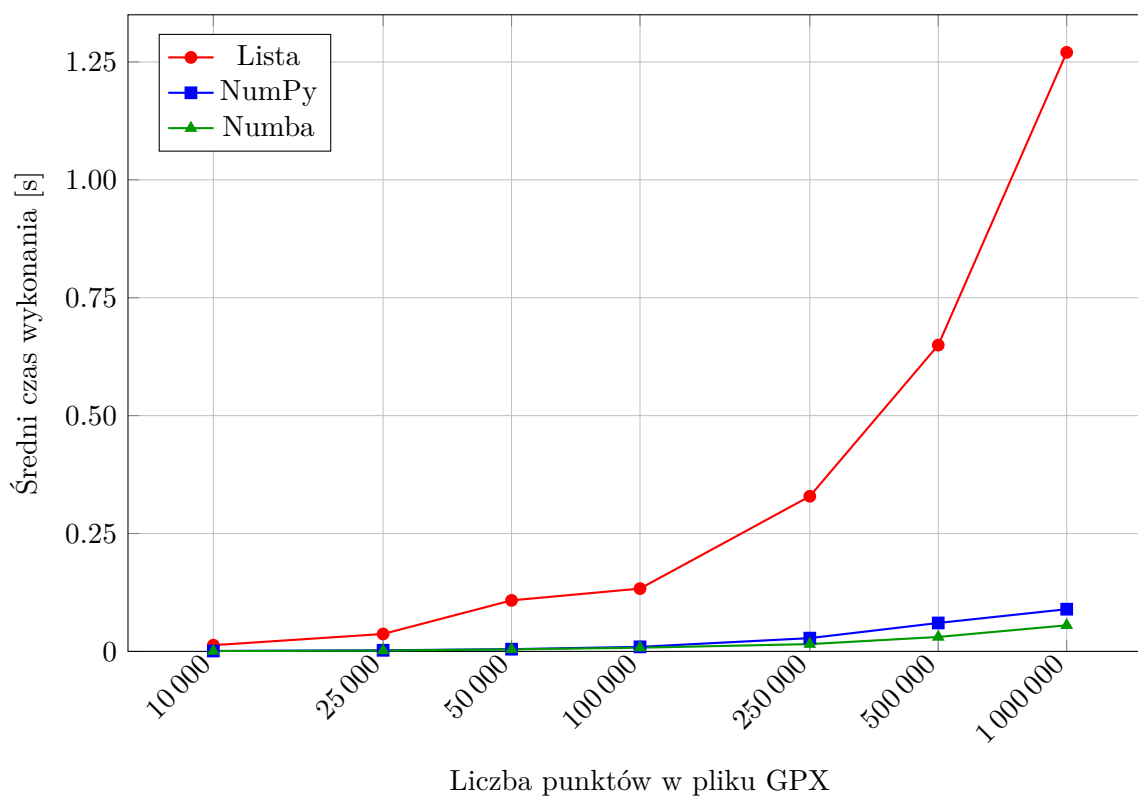
Pierwszą analizowaną operacją jest obliczanie całkowitego dystansu trasy na podstawie kolejnych par punktów śladu. Jest to operacja liniowa względem liczby punktów wejściowych, dlatego stanowi dobry punkt odniesienia dla porównania wariantu listowego, NumPy i Numba.

Tabela 4.3 przedstawia średnie czasy wykonania uzyskane dla kolejnych rozmiarów danych wejściowych. W każdym przypadku porównywano te same pliki GPX, a różnice dotyczyły wyłącznie sposobu realizacji obliczeń.

Tabela 4.3: Średnie czasy wykonania benchmarku obliczania dystansu

Liczba punktów	Lista [s]	NumPy [s]	Numba [s]
10 000	0.013216	0.000891	0.000714
25 000	0.036973	0.002391	0.001820
50 000	0.108215	0.004717	0.004384
100 000	0.133220	0.009641	0.007873
250 000	0.328946	0.028175	0.015503
500 000	0.649536	0.060318	0.030652
1 000 000	1.270193	0.089529	0.055405

Porównanie czasu obliczania dystansu



Wykres 4.1: Porównanie średniego czasu obliczania dystansu dla wariantów listowego, NumPy i Numba.

Na podstawie danych z tabeli 4.3 można stwierdzić, że wszystkie trzy warianty wykazują wzrost czasu wykonania wraz ze wzrostem liczby punktów wejściowych, co jest zgodne z liniowym charakterem analizowanej operacji. Jednocześnie różnice między wariantami są wyraźne w całym badanym zakresie danych.

Wariant listowy okazał się najwolniejszy dla wszystkich rozmiarów wejścia. Wariant NumPy był wyraźnie szybszy od wariantu listowego, natomiast najlepsze wyniki w

całym zakresie danych uzyskał wariant Numba. Dla najmniejszego testowanego pliku, zawierającego 10 000 punktów, czas wykonania zmniejszył się z około 0.0132 s dla wariantu listowego do około 0.00071 s dla wariantu Numba. Dla największego pliku, zawierającego 1 000 000 punktów, czasy wynosiły odpowiednio około 1.27 s, 0.0895 s oraz 0.0554 s.

Oznacza to, że w przeprowadzonym eksperymencie wariant NumPy był około 14 razy szybszy od wariantu listowego dla największego badanego pliku, natomiast wariant Numba był od niego szybszy około 23 razy. Wynik ten sugeruje, że dla operacji polegającej na wielokrotnym wykonywaniu prostych obliczeń numerycznych na kolejnych parach punktów zarówno operacje tablicowe, jak i kompilacja JIT przynoszą istotną korzyść wydajnościową.

Należy jednak ostrożnie interpretować przyczyny uzyskanego przyspieszenia. Jedną z możliwych przyczyn przewagi wariantu NumPy może być ograniczenie kosztu jawnych pętli Pythona przez zastosowanie operacji tablicowych. Z kolei w przypadku wariantu Numba znaczenie może mieć przeniesienie pętli numerycznej do funkcji kompilowanej. Na podstawie samych benchmarków czasowych nie należy jednak jednoznacznie przypisywać całego przyspieszenia jednej przyczynie. Wynik zależy zarówno od sposobu zapisu pętli, jak i od reprezentacji danych oraz kosztu wywołań funkcji numerycznych.

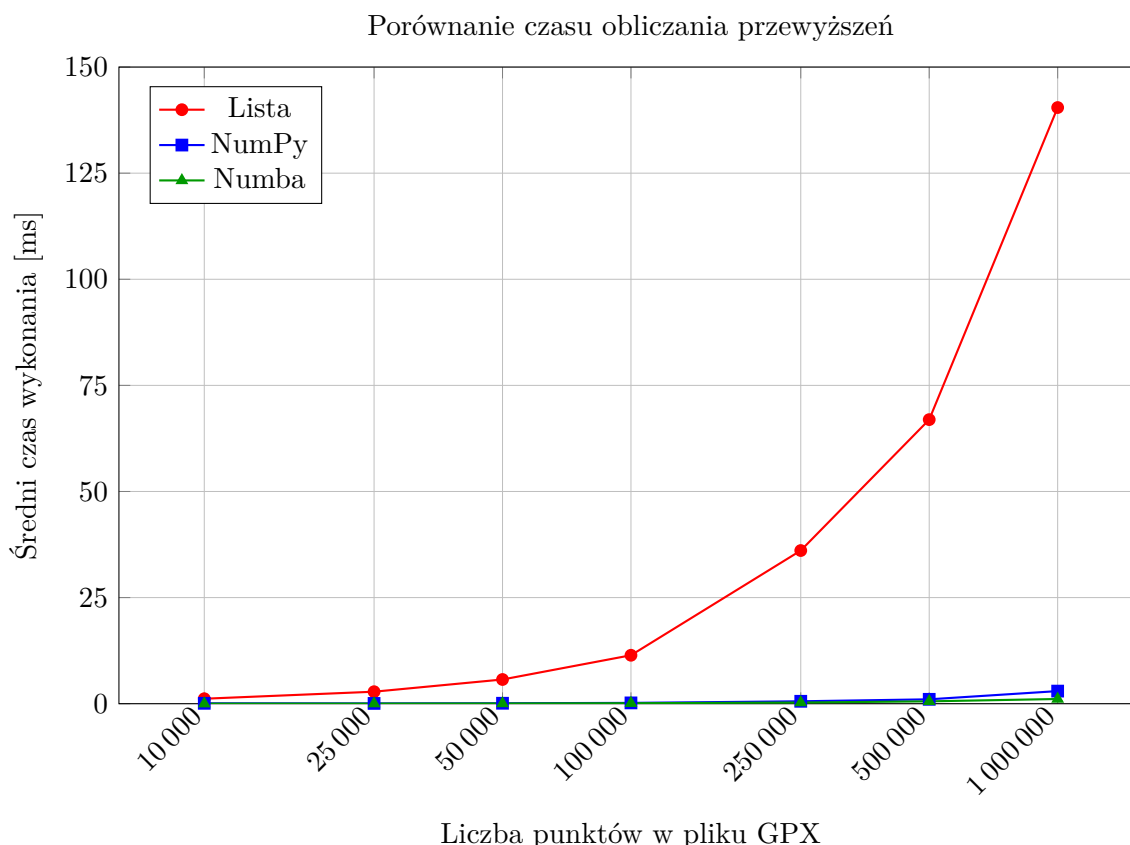
4.3.2 Obliczanie przewyższeń

Drugą analizowaną operacją jest obliczanie sumy podejść i zejść na podstawie kolejnych wartości wysokości. W odróżnieniu od obliczania dystansu operacja ta nie wymaga funkcji trygonometrycznych ani przeliczania współrzędnych geograficznych. Algorytm opiera się na wyznaczeniu różnic wysokości między sąsiednimi punktami oraz oddzielnym zsumowaniu różnic dodatnich i ujemnych.

Tabela 4.4 przedstawia średnie czasy wykonania uzyskane dla wariantu listowego, NumPy oraz Numba. Podobnie jak w przypadku dystansu, pomiar wykonano dla plików o liczbie punktów od 10 000 do 1 000 000.

Tabela 4.4: Średnie czasy wykonania benchmarku obliczania przewyższeń

Liczba punktów	Lista [s]	NumPy [s]	Numba [s]
10 000	0.001171	0.000090	0.000011
25 000	0.002825	0.000066	0.000027
50 000	0.005702	0.000104	0.000054
100 000	0.011412	0.000189	0.000113
250 000	0.036085	0.000566	0.000273
500 000	0.066933	0.001030	0.000554
1 000 000	0.140462	0.002967	0.001107



Wykres 4.2: Porównanie średniego czasu obliczania przewyższeń dla wariantów listowego, NumPy i Numba. Czas przedstawiono w milisekundach.

Na podstawie tabeli 4.4 można zauważyć, że wariant listowy wykazuje wzrost czasu wykonania wraz ze wzrostem liczby punktów wejściowych. Jest to zgodne z charakterem operacji, ponieważ funkcja przechodzi po kolejnych parach sąsiednich wysokości i dla każdej z nich oblicza różnicę.

Warianty NumPy i Numba osiągnęły znacznie krótsze czasy wykonania niż wariant listowy. Dla największego pliku, zawierającego 1 000 000 punktów, wariant listowy wykonał obliczenia w czasie około 0.1405 s, wariant NumPy w czasie około 0.0030 s, a wariant Numba w czasie około 0.0011 s. Oznacza to, że dla tego przypadku wariant NumPy był około 47 razy szybszy od wariantu listowego, natomiast wariant Numba około 127 razy szybszy od wariantu listowego.

Różnice czasu można wyjaśnić sposobem wykonania obliczeń. Wariant NumPy oblicza tablicę różnic wysokości za pomocą działania na przesuniętych fragmentach tablicy, a następnie stosuje maski logiczne do wyboru wartości dodatnich i ujemnych. Wariant Numba zachowuje prostą pętlę po kolejnych wysokościach, ale wykonuje ją w funkcji skompilowanej. Dla tak prostej operacji numerycznej narzut pętli Pythona w wariantcie listowym staje się dobrze widoczny już przy większych zbiorach danych.

Należy jednak ostrożnie interpretować wyniki dla najmniejszych plików. Czasy wy-

konania wariantów NumPy i Numba są tam bardzo małe, dlatego pojedyncze różnice mogą wynikać z narzutu wywołania funkcji, chwilowego obciążenia systemu lub innych czynników pomiarowych. Z tego powodu większe znaczenie interpretacyjne ma ogólny trend widoczny dla rosnącej liczby punktów niż pojedyncze odchylenia między sąsiednimi rozmiarami danych.

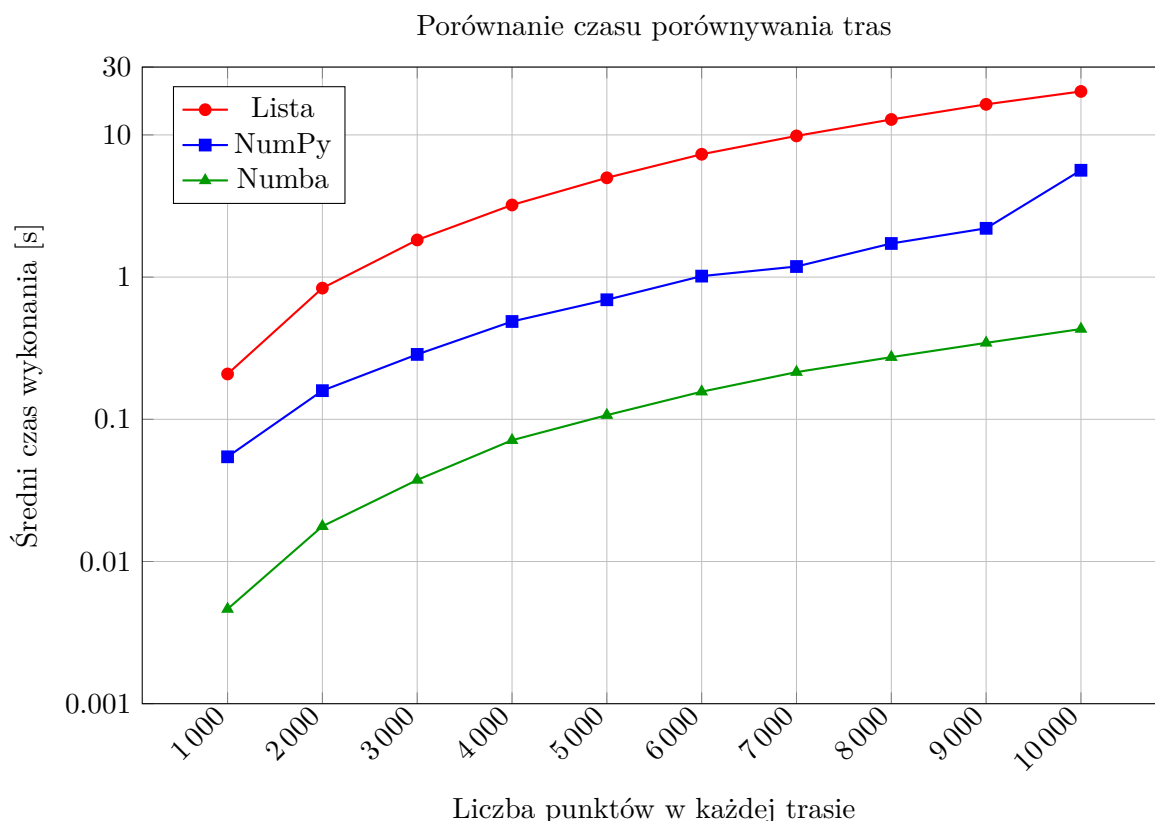
4.3.3 Porównywanie trasy zawodnika z trasą referencyjną

Kolejną analizowaną operacją jest porównywanie trasy zawodnika z trasą referencyjną. W tej metodzie każdy punkt śladu zawodnika jest porównywany z segmentami trasy referencyjnej. Oznacza to, że koszt obliczeniowy nie rośnie liniowo względem liczby punktów, lecz zależy od liczby relacji punkt–segment. Dla par tras o tej samej liczbie punktów liczba takich sprawdzeń wynosi w przybliżeniu $n(n - 1)$, gdzie n oznacza liczbę punktów trasy.

Tabela 4.5 przedstawia średnie czasy wykonania dla wariantu listowego, NumPy oraz Numba. W benchmarku wykorzystano pary plików `trasa_N.gpx` oraz `trasa_N_zawodnik.gpx`, gdzie liczba punktów zwiększała się od 1000 do 10 000.

Tabela 4.5: Średnie czasy wykonania benchmarku porównywania tras

Liczba punktów	Relacje punkt–segment	Lista [s]	NumPy [s]	Numba [s]
1 000	999 000	0.208341	0.054492	0.004629
2 000	3 998 000	0.836794	0.159032	0.017689
3 000	8 997 000	1.825742	0.285992	0.037458
4 000	15 996 000	3.215168	0.487333	0.071246
5 000	24 995 000	4.992853	0.692888	0.106967
6 000	35 994 000	7.311315	1.016197	0.156377
7 000	48 993 000	9.839682	1.186977	0.214730
8 000	63 992 000	12.840634	1.723951	0.273499
9 000	80 991 000	16.407631	2.203830	0.344766
10 000	99 990 000	20.216993	5.641450	0.431464



Wykres 4.3: Porównanie średniego czasu porównywania tras dla wariantów listowego, NumPy i Numba. Oś czasu przedstawiono w skali logarytmicznej.

Wyniki przedstawione w tabeli 4.5 pokazują znacznie szybszy wzrost czasu wykonania niż w przypadku operacji wykonywanych na pojedynczej trasie. Jest to zgodne ze strukturą metody, ponieważ zwiększenie liczby punktów powoduje wzrost liczby relacji punkt–segment. Dla 1000 punktów liczba sprawdzeń wynosi 999 000, natomiast dla 10 000 punktów wzrasta do 99 990 000.

Wariant listowy uzyskał najwyższe czasy wykonania w całym zakresie danych. Dla pary tras zawierających po 1000 punktów średni czas wyniósł około 0.208 s, natomiast dla 10 000 punktów wzrósł do około 20.217 s. Wariant NumPy był szybszy od wariantu listowego, ale nie w takim stopniu jak przy obliczaniu dystansu lub przewyższeń. Dla 10 000 punktów czas wykonania wariantu NumPy wyniósł około 5.641 s.

Najkrótsze czasy wykonania uzyskał wariant Numba. Dla 1000 punktów średni czas wyniósł około 0.0046 s, a dla 10 000 punktów około 0.4315 s. Oznacza to, że dla największego badanego przypadku wariant Numba był około 47 razy szybszy od wariantu listowego oraz około 13 razy szybszy od wariantu NumPy.

Różnice między wariantami można powiązać ze strukturą zastosowanych funkcji. Wariant NumPy wykonuje część obliczeń tablicowo, ale nadal iteruje po kolejnych punktach śladu zawodnika. Częściowa wektoryzacja dotyczy przede wszystkim obliczania odległości jednego punktu od wszystkich segmentów trasy referencyjnej. Wariant Numba zachowuje

strukturę pętli, lecz przenosi ich wykonanie do funkcji kompilowanej. W przypadku metody wymagającej dużej liczby powtarzalnych obliczeń punkt–segment taka organizacja może być jedną z przyczyn wyraźnego skrócenia czasu wykonania.

Warto zwrócić uwagę, że dla wariantu NumPy przy 10 000 punktów widoczny jest większy wzrost czasu niż między wcześniejszymi rozmiarami danych. Może to wynikać między innymi z kosztu tworzenia tablic pośrednich dla coraz większej liczby segmentów, wpływu zarządzania pamięcią lub chwilowych czynników pomiarowych. Nie należy jednak interpretować tego pojedynczego wzrostu jako samodzielnego dowodu na konkretną przyczynę. Może on wynikać zarówno ze struktury obliczeń, jak i z kosztu tworzenia tablic pośrednich lub z czynników środowiskowych wpływających na pomiar czasu.

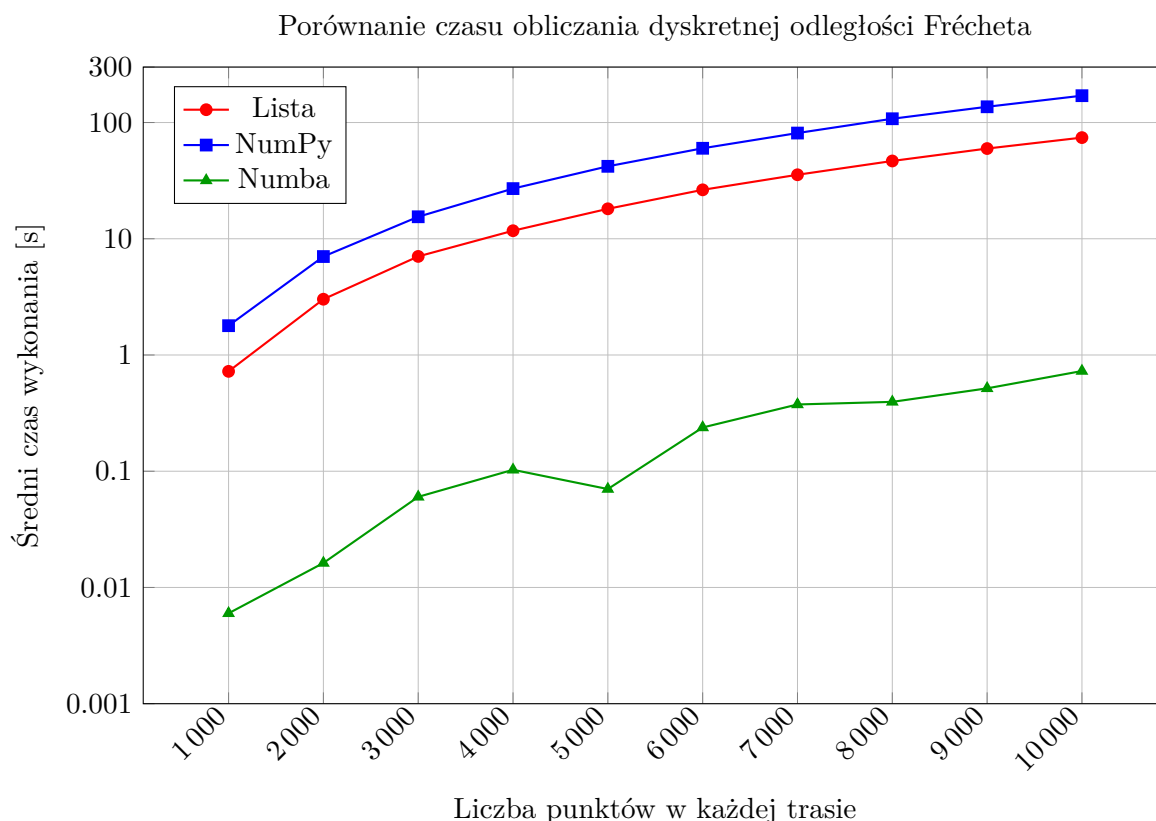
4.3.4 Dyskretna odległość Frécheta

Ostatnią analizowaną operacją jest obliczanie dyskretnej odległości Frécheta między trasą referencyjną a trasą zawodnika. W odróżnieniu od metody punkt–segment, która dla każdego punktu zawodnika szuka najbliższego segmentu trasy referencyjnej, metoda Frécheta porównuje dwie sekwencje punktów z uwzględnieniem ich kolejności. W przyjętej implementacji wymaga to utworzenia macierzy programowania dynamicznego, której liczba komórek jest iloczynem liczby punktów obu tras.

Tabela 4.6 przedstawia średnie czasy wykonania dla wariantu listowego, NumPy oraz Numba. Zastosowano ten sam zakres par tras jak w benchmarku porównywania punktów z segmentami, czyli od 1000 do 10 000 punktów.

Tabela 4.6: Średnie czasy wykonania benchmarku dyskretnej odległości Frécheta

Liczba punktów	Komórki DP	Lista [s]	NumPy [s]	Numba [s]
1 000	1 000 000	0.723205	1.787297	0.005993
2 000	4 000 000	3.018974	7.030743	0.016234
3 000	9 000 000	7.048458	15.460332	0.060198
4 000	16 000 000	11.724797	27.014392	0.102872
5 000	25 000 000	18.100327	41.999846	0.070250
6 000	36 000 000	26.357626	60.041088	0.237831
7 000	49 000 000	35.527144	81.173815	0.375307
8 000	64 000 000	46.686739	107.765870	0.395133
9 000	81 000 000	59.718604	136.604679	0.516438
10 000	100 000 000	74.181451	170.175190	0.727838



Wykres 4.4: Porównanie średniego czasu obliczania dyskretnej odległości Fréchet dla wariantów listowego, NumPy i Numba. Oś czasu przedstawiono w skali logarytmicznej.

Wyniki przedstawione w tabeli 4.6 pokazują bardzo szybki wzrost czasu wykonania wraz ze wzrostem liczby punktów. Jest to zgodne ze strukturą algorytmu, ponieważ dla tras o tej samej liczbie punktów liczba komórek macierzy programowania dynamicznego rośnie kwadratowo. Dla 1000 punktów macierz zawiera 1 000 000 komórek, natomiast dla 10 000 punktów liczba ta wzrasta do 100 000 000. Wariant listowy dla największego przypadku testowego wykonał obliczenia w czasie około 74.18 s. Wariant NumPy był w tej metodzie wolniejszy i dla 10 000 punktów osiągnął średni czas około 170.18 s. Wynik ten można wyjaśnić sposobem użycia NumPy w tej implementacji. Biblioteka służy tu głównie do przechowywania macierzy jako tablicy `ndarray`, natomiast kolejne komórki macierzy są nadal wypełniane iteracyjnie w zagnieżdżonych pętlach zapisanych w Pythonie.

Najkrótsze czasy wykonania uzyskał wariant Numba. Dla 1000 punktów średni czas wyniósł około 0.006 s, natomiast dla 10 000 punktów około 0.728 s. Oznacza to, że dla największego badanego przypadku wariant Numba był około 102 razy szybszy od wariantu listowego oraz około 234 razy szybszy od wariantu NumPy.

Szczególnie istotne jest porównanie wariantu listowego i NumPy. W operacjach takich jak obliczanie dystansu lub przewyższeń NumPy pozwalał zapisać znaczną część obliczeń jako operacje tablicowe. W przypadku dyskretnej odległości Fréchet zależność rekurencyjna wymusza jednak sekwencyjne wypełnianie macierzy. Z tego powodu samo zastosowanie

tablicy NumPy nie gwarantuje przyspieszenia. Może wręcz zwiększyć koszt wykonania, jeżeli dostęp do elementów tablicy odbywa się w zagnieżdżonej pętli zapisanej nadal po stronie Pythona.

Wariant Numba lepiej odpowiada strukturze tej metody, ponieważ zachowuje jawne pętle, ale wykonuje je w funkcji skompilowanej. Wyniki benchmarku wskazują, że dla algorytmu programowania dynamicznego, w którym pełna wektoryzacja jest utrudniona, kompilacja numerycznego rdzenia może być znacznie korzystniejsza niż samo użycie tablic NumPy.

Warto zauważyć, że czasy wariantu Numba nie rosną idealnie monotonicznie dla każdego kolejnego rozmiaru danych. Przykładowo wynik dla 5000 punktów jest niższy niż dla 4000 punktów. Może to wynikać z czynników pomiarowych, takich jak chwilowe obciążenie systemu, zarządzanie pamięcią, sposób alokacji macierzy lub wpływ środowiska wykonania. Z tego powodu pojedyncze odchylenia nie powinny być interpretowane jako cecha samego algorytmu. Istotniejszy jest ogólny trend, zgodnie z którym czas wykonania rośnie wraz z liczbą komórek macierzy DP, a wariant Numba pozostaje wyraźnie najszybszy w całym badanym zakresie.

4.4 Wnioski

Wyniki benchmarków pokazują, że wpływ reprezentacji danych oraz mechanizmu wykonania kodu zależy od charakteru analizowanej operacji. Operacje liniowe, takie jak obliczanie dystansu oraz przewyższeń, zachowują się inaczej niż metody porównujące dwie trasy, których koszt rośnie wraz z liczbą relacji punkt-segment albo liczbą komórek macierzy programowania dynamicznego.

W przypadku obliczania dystansu i przewyższeń warianty NumPy oraz Numba uzyskały wyraźnie krótsze czasy wykonania niż wariant listowy. Wynika to z faktu, że obie operacje można wyrazić jako przetwarzanie sąsiednich elementów jednej sekwencji. Wariant NumPy wykorzystuje w tych przypadkach przesunięte wycinki tablic oraz operacje tablicowe, natomiast wariant Numba zachowuje jawną pętlę, ale wykonuje ją w funkcji skompilowanej. Dla takich operacji ograniczenie narzutu interpretera Pythona może być jedną z istotnych przyczyn uzyskanego przyspieszenia.

Inaczej wygląda sytuacja w przypadku metod porównujących dwie trasy. Porównywanie punktów z segmentami trasy referencyjnej ma większy koszt obliczeniowy niż operacje liniowe, ponieważ każdy punkt śladu zawodnika jest porównywany z wieloma segmentami trasy referencyjnej. W tej metodzie wariant NumPy był szybszy od wariantu listowego, ale jego przewaga była mniejsza niż przy dystansie i przewyższeniach. Jest to zgodne z implementacją, ponieważ wariant NumPy nie eliminuje całej pętli po punktach zawodnika. Wektoryzacji podlega przede wszystkim obliczenie odległości jednego punktu od wszystkich

segmentów referencyjnych.

Najbardziej wyraźne ograniczenie wariantu NumPy zaobserwowano przy dyskretnej odległości Frécheta. W tej metodzie NumPy służy głównie do przechowywania macierzy programowania dynamicznego jako tablicy `ndarray`, natomiast sama macierz nadal jest wypełniana w zagnieżdżonych pętlach zapisanych w Pythonie. W przeprowadzonych pomiarach wariant NumPy był wolniejszy od wariantu listowego. Wynik ten pokazuje, że samo użycie tablic NumPy nie jest równoznaczne z przyspieszeniem obliczeń. Korzyść z NumPy zależy od tego, czy struktura algorytmu pozwala przenieść znaczącą część pracy do operacji tablicowych wykonywanych poza jawną pętlę Pythona.

Wariant Numba uzyskał najkrótsze czasy wykonania we wszystkich analizowanych grupach operacji. Szczególnie duża przewaga wystąpiła w metodach zawierających kosztowne pętle numeryczne, takich jak porównywanie tras oraz dyskretna odległość Frécheta. W tych przypadkach struktura algorytmu jest trudna do pełnej wektoryzacji, ale dobrze nadaje się do kompilacji pętli operujących na liczbach i tablicach. Wyniki te wskazują, że dla analizowanych metod Numba była najkorzystniejszym wariantem wykonawczym, jednak wniosek ten dotyczy konkretnej implementacji, danych testowych oraz środowiska pomiarowego.

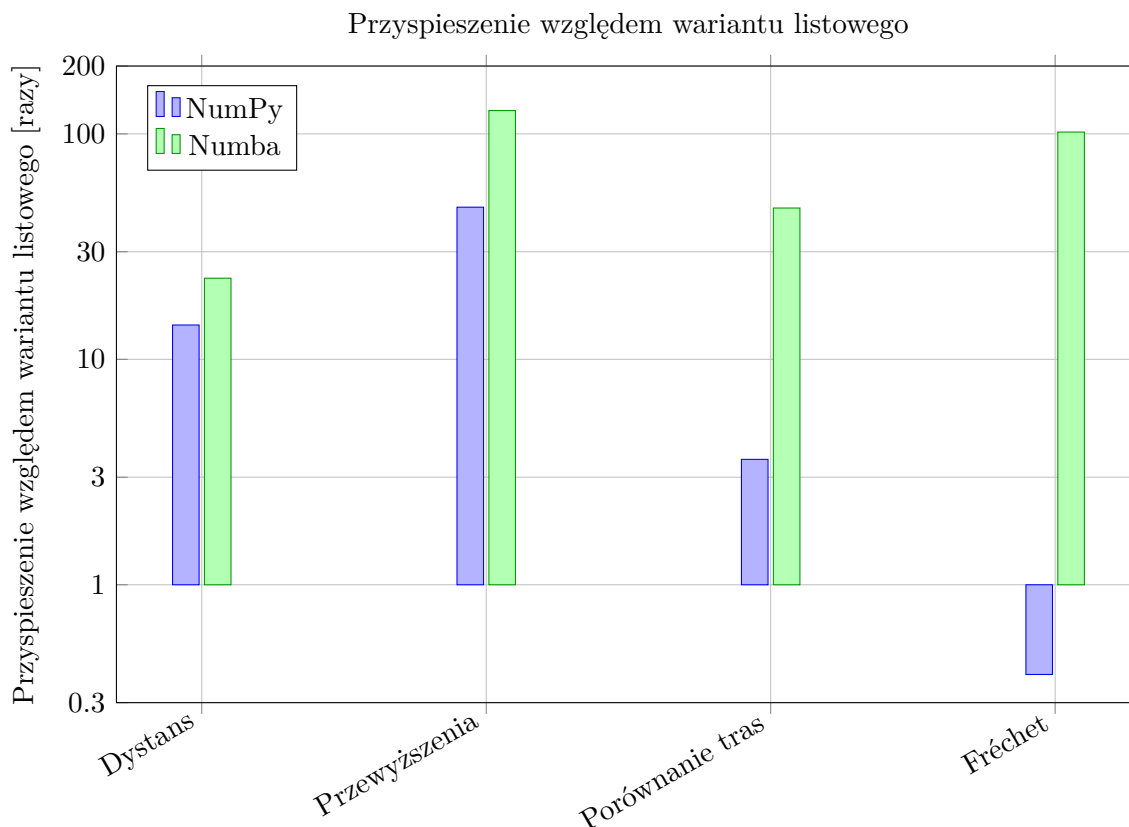
Tabela 4.7: Stosunek czasu wariantu listowego do czasu wariantów NumPy i Numba dla największych badanych danych

Operacja	Lista / NumPy	Lista / Numba
Obliczanie dystansu, 1 000 000 punktów	około 14.2×	około 22.9×
Obliczanie przewyższeń, 1 000 000 punktów	około 47.3×	około 126.9×
Porównywanie tras, 10 000 punktów	około 3.6×	około 46.9×
Dyskretna odległość Fréche- ta, 10 000 punktów	około 0.44×	około 101.9×

Wartość większa od 1 oznacza, że dany wariant był szybszy od wariantu listowego. Wartość mniejsza od 1, widoczna dla wariantu NumPy przy dyskretnej odległości Frécheta, oznacza wynik wolniejszy od wariantu listowego.

Tabela 4.7 zbiera najważniejsze różnice między wariantami dla największych danych użytych w poszczególnych benchmarkach. Zestawienie to pokazuje, że przyspieszenie nie jest stałą cechą biblioteki lub technologii, lecz zależy od konkretnej operacji. NumPy bardzo dobrze sprawdziło się przy przewyższeniach, gdzie obliczenia można sprowadzić do różnic sąsiednich elementów i masek logicznych. Jednocześnie przy Fréchetie wariant NumPy był wolniejszy od listowego, ponieważ nie usuwał głównej zależności rekurencyjnej

ani zagnieżdżonych pętli.



Wykres 4.5: Przybliżone przyspieszenie wariantów NumPy i Numba względem wariantu listowego dla największych danych w każdej grupie operacji. Oś pionowa została przedstawiona w skali logarytmicznej.

Interpretując wyniki, należy uwzględnić ograniczenia przyjętej metodyki. Dane wejściowe miały charakter syntetyczny, dzięki czemu możliwe było kontrolowanie liczby punktów, kompletności struktury GPX oraz powtarzalności benchmarków. Nie oznacza to jednak, że odwzorowują one wszystkie cechy rzeczywistych śladów GPS, takich jak przerwy w zapisie, zmienny interwał rejestracji, błędy wysokości czy zakłócenia pomiaru pozycji.

Na wyniki wpływa również sposób wydzielenia mierzonego fragmentu programu. Benchmarki obejmowały czas wykonania funkcji analitycznych na przygotowanych danych, a nie pełny czas przetwarzania pliku od odczytu XML do uzyskania wyniku końcowego. Takie rozdzielanie było potrzebne, ponieważ celem pracy było porównanie wariantów obliczeniowych, a nie wydajności parsera GPX.

Pomiary czasu mogą być zależne od chwilowego obciążenia systemu, zarządzania pamięcią, środowiska WSL oraz mechanizmów działania interpretera Pythona. Ma to szczególne znaczenie dla bardzo krótkich czasów wykonania, gdzie niewielkie zakłócenia mogą wpływać na pojedyncze przebiegi. Z tego powodu interpretacja wyników powinna opierać się przede wszystkim na trendach widocznych dla rosnącej liczby punktów, a nie

na pojedynczych odchyleniach.

W przypadku wariantu Numba trzeba dodatkowo pamiętać o rozdzieleniu kosztu kompilacji JIT od czasu wykonania funkcji już skompilowanej. W benchmarkach zastosowano wywołania przygotowawcze, dlatego prezentowane wyniki opisują przede wszystkim czas działania funkcji po zakończeniu etapu kompilacji.

4.5 Podsumowanie

Uzyskane wyniki pokazują, że wybór wariantu implementacyjnego ma istotny wpływ na czas wykonania metod analizy tras GPX. Największe różnice wystąpiły dla metod zawierających dużą liczbę powtarzalnych operacji numerycznych. Wariant listowy był najwolniejszy we wszystkich benchmarkach poza porównaniem z NumPy dla metody Fréchet'a, gdzie wariant NumPy okazał się jeszcze wolniejszy. Wynik ten podkreśla, że wykorzystanie biblioteki NumPy powinno być uzasadnione strukturą algorytmu, a nie samym faktem pracy na danych liczbowych.

Najbardziej uniwersalnie korzystny okazał się wariant Numba, szczególnie w operacjach z trudną do zwektoryzowania strukturą pętli. Nie oznacza to jednak, że Numba powinna być traktowana jako rozwiązanie gwarantujące przyspieszenie w każdym przypadku. Uzyskane wyniki należy odnosić do konkretnego kodu, danych wejściowych, liczby punktów oraz środowiska testowego. Wnioski z benchmarków potwierdzają główne założenie pracy: wydajność programu do analizy tras GPX zależy nie tylko od zastosowanego algorytmu, lecz także od reprezentacji danych i mechanizmu wykonania kodu.

Bibliografia

- [1] Ride with GPS, *The Elevation Profile on Web*, tryb dostępu: <https://support.ridewithgps.com/hc/en-us/articles/4419005868315>, (dostęp: 21 maja 2026 r.).
- [2] Ride with GPS, *Route Planning 101*, tryb dostępu: <https://support.ridewithgps.com/hc/en-us/articles/4415462488475>, (dostęp: 21 maja 2026 r.).
- [3] Topografix, *GPX 1.1 Schema Documentation*, tryb dostępu: <https://www.topografix.com/gpx/1/1/>, (dostęp: 2 marca 2026 r.).
- [4] J. L. Hennessy, D. A. Patterson, *Computer Architecture: A Quantitative Approach*, 6th ed., Morgan Kaufmann, Cambridge, 2017.
- [5] R. E. Bryant, D. R. O'Hallaron, *Computer Systems: A Programmer's Perspective*, 3rd ed., Pearson, Boston, 2016.
- [6] Python Software Foundation, *The Python Language Reference: Data Model*, tryb dostępu: <https://docs.python.org/3/reference/datamodel.html>, (dostęp: 3 marca 2026 r.).
- [7] Python Software Foundation, *Design and History FAQ: How Are Lists Implemented in CPython?*, tryb dostępu: <https://docs.python.org/3/faq/design.html>, (dostęp: 21 maja 2026 r.).
- [8] Python Software Foundation, *xml.etree.ElementTree – The ElementTree XML API*, tryb dostępu: <https://docs.python.org/3/library/xml.etree.elementtree.html>, (dostęp: 21 maja 2026 r.).
- [9] pyproj Developers, *Transformer – pyproj Documentation*, tryb dostępu: <https://pyproj4.github.io/pyproj/stable/api/transformer.html>, (dostęp: 21 maja 2026 r.).
- [10] M. J. Flynn, *Very High-Speed Computing Systems, Proceedings of the IEEE*, 1966, t. 54, nr 12, s. 1901–1909, doi: 10.1109/PROC.1966.5273.

- [11] OpenMP Architecture Review Board, *OpenMP API Specification 5.1 – SIMD Construct*, tryb dostępu: <https://www.openmp.org/spec-html/5.1/openmpsu49.html>, (dostęp: 2 marca 2026 r.).
- [12] G. M. Amdahl, *Validity of the Single Processor Approach to Achieving Large Scale Computing Capabilities*, *AFIPS Conference Proceedings*, 1967, t. 30, s. 483–485, doi: 10.1145/1465482.1465560.
- [13] J. Januszewski, *Systemy satelitarne GPS, Galileo i inne*, Wydawnictwo Naukowe PWN, Warszawa, 2010.
- [14] National Geospatial-Intelligence Agency, *Department of Defense World Geodetic System 1984: Its Definition and Relationships with Local Geodetic Systems*, NGA.STND.0036_1.0.0_WGS84, Version 1.0.0, 2014.
- [15] J. Pasławski (red.), *Wprowadzenie do kartografii i topografii*, Nowa Era, Warszawa–Wrocław, 2010.
- [16] EPSG.io, *EPSG:2180 – ETRF2000-PL / CS92*, tryb dostępu: <https://epsg.io/2180>, (dostęp: 21 maja 2026 r.).
- [17] R. Sanchez, P. E. Egli, K. Jornet, M. Duggan, M. Besomi, *How Fractal Complexity Distorts Distance and Elevation Gain in Trail and Mountain Running: The Case for Course Measurement Standardization*, *Proceedings of the Institution of Mechanical Engineers, Part P: Journal of Sports Engineering and Technology*, 2025, doi: 10.1177/17543371251341660.
- [18] R. W. Sinnott, *Virtues of the Haversine*, *Sky and Telescope*, 1984, t. 68, nr 2, s. 159.
- [19] P. Ranacher, R. Brunauer, W. Trutschnig, S. Van der Spek, S. Reich, *Why GPS Makes Distances Bigger Than They Are*, *International Journal of Geographical Information Science*, 2016, t. 30, nr 2, s. 316–333, doi: 10.1080/13658816.2015.1086924.
- [20] Open Geospatial Consortium, *OpenGIS Implementation Standard for Geographic Information – Simple Feature Access – Part 1: Common Architecture*, OGC 06-103r4, version 1.2.1, 2011, tryb dostępu: <https://docs.ogc.org/is/06-103r4/06-103r4.pdf>, (dostęp: 21 maja 2026 r.).
- [21] C. Ericson, *Real-Time Collision Detection*, Morgan Kaufmann, San Francisco, 2005, rozdz. 5.1.2: *Closest Point on Line Segment to Point*, ISBN 978-1-55860-732-3.
- [22] T. Eiter, H. Mannila, *Computing Discrete Fréchet Distance*, Technical Report CD-TR 94/64, Christian Doppler Laboratory for Expert Systems, Technische Universität

- Wien, 1994, tryb dostępu: <https://www.kr.tuwien.ac.at/staff/eiter/et-archive/files/cdtr9464.pdf>, (dostęp: 17 maja 2026 r.).
- [23] M. Gorelick, I. Ozsvald, *High Performance Python: Practical Performant Programming for Humans*, 2nd ed., O'Reilly Media, Sebastopol, 2020.
- [24] C. R. Harris, K. J. Millman, S. J. van der Walt, R. Gommers, P. Virtanen, D. Cournapeau, E. Wieser, J. Taylor, S. Berg, N. J. Smith i in., *Array Programming with NumPy*, *Nature*, 2020, t. 585, s. 357–362, doi: 10.1038/s41586-020-2649-2.
- [25] NumPy Developers, *The N-dimensional Array*, tryb dostępu: <https://numpy.org/doc/stable/reference/arrays.ndarray.html>, (dostęp: 21 maja 2026 r.).
- [26] NumPy Developers, *Universal Functions (ufunc)*, tryb dostępu: <https://numpy.org/doc/stable/reference/ufuncs.html>, (dostęp: 21 maja 2026 r.).
- [27] NumPy Developers, *Broadcasting*, tryb dostępu: <https://numpy.org/doc/stable/user/basics.broadcasting.html>, (dostęp: 21 maja 2026 r.).
- [28] S. K. Lam, A. Pitrou, S. Seibert, *Numba: A LLVM-Based Python JIT Compiler*, *Proceedings of the Second Workshop on the LLVM Compiler Infrastructure in HPC*, 2015, s. 1–6, doi: 10.1145/2833157.2833162.
- [29] Numba Developers, *Compiling Python Code with @jit*, tryb dostępu: <https://numba.readthedocs.io/en/stable/user/jit.html>, (dostęp: 21 maja 2026 r.).

Spis listingów

1.1	Fragment przykładowego pliku GPX	11
3.1	Ogólna struktura katalogów projektu	25
3.2	Parser GPX w wariancie listowym	28
3.3	Parser GPX w wariancie NumPy	29
3.4	Obliczanie dystansu i przewyższeń w wariancie listowym	31
3.5	Obliczanie dystansu i przewyższeń w wariancie NumPy	32
3.6	Obliczanie dystansu i przewyższeń w wariancie Numba	33
3.7	Funkcje pomocnicze porównywania tras w wariancie listowym	36
3.8	Agregacja wyników porównania tras w wariancie listowym	37
3.9	Budowa segmentów i odległość punkt-segment w wariancie NumPy	38
3.10	Porównywanie tras w wariancie NumPy	39
3.11	Rdzeń porównywania tras w wariancie Numba	41
3.12	Funkcja zewnętrzna porównywania tras w wariancie Numba	42
3.13	Dyskretna odległość Frécheta w wariancie listowym	44
3.14	Dyskretna odległość Frécheta w wariancie NumPy	45
3.15	Rdzeń obliczania dyskretnej odległości Frécheta w wariancie Numba	46
3.16	Funkcja zewnętrzna obliczania odległości Frécheta w wariancie Numba	47
3.17	Funkcja pomocnicza do pomiaru czasu wykonania	48
3.18	Zakres danych i liczba pomiarów dla operacji liniowych	49
3.19	Zakres danych dla benchmarków metod porównujących dwie trasy	50
3.20	Przykład krótkiego wywołania wymuszającego kompilację Numby	51
3.21	Przykładowa struktura rekordu wynikowego benchmarku	51

Spis wykresów

4.1	Porównanie średniego czasu obliczania dystansu dla wariantów listowego, NumPy i Numba.	56
4.2	Porównanie średniego czasu obliczania przewyższeń dla wariantów listowego, NumPy i Numba. Czas przedstawiono w milisekundach.	58
4.3	Porównanie średniego czasu porównywania tras dla wariantów listowego, NumPy i Numba. Oś czasu przedstawiono w skali logarytmicznej.	60
4.4	Porównanie średniego czasu obliczania dyskretnej odległości Frécheta dla wariantów listowego, NumPy i Numba. Oś czasu przedstawiono w skali logarytmicznej.	62
4.5	Przybliżone przyspieszenie wariantów NumPy i Numba względem wariantu listowego dla największych danych w każdej grupie operacji. Oś pionowa została przedstawiona w skali logarytmicznej.	65

Spis tabel

4.1	Podstawowe parametry środowiska testowego	54
4.2	Zakres danych i liczba powtórzeń w benchmarkach	55
4.3	Średnie czasy wykonania benchmarku obliczania dystansu	56
4.4	Średnie czasy wykonania benchmarku obliczania przewyższeń	57
4.5	Średnie czasy wykonania benchmarku porównywania tras	59
4.6	Średnie czasy wykonania benchmarku dyskretnej odległości Frécheta . . .	61
4.7	Stosunek czasu wariantu listowego do czasu wariantów NumPy i Numba dla największych badanych danych	64

